A Comprehensive Survey on Security Challenges in Large Language Models

August 2026

Abstract

Large language models (LLMs) have become integral to modern software-development workflows, automating tasks from code generation to bug detection with human-competitive performance. However, this rapid integration into critical applications introduces substantial security vulnerabilities, as the instruction-following capability that enables flexibility also exposes models to attacks capable of corrupting outputs, exfiltrating data, and compromising downstream systems. This survey presents a systematic literature review of the emerging field of LLM security, synthesizing findings from 117 cited sources after a primary-source attribution audit across the principal threat categories: adversarial attacks, data and model poisoning, prompt injection, and hallucination-induced integrity failures. We organize the landscape into a taxonomy spanning training-time and inference-time threats, analyzing mechanisms ranging from backdoor poisoning and jailbreak exploits to multi-turn conversational manipulation and automated red-teaming frameworks. The survey further examines the critical reconceptualization of hallucination—long treated as an accuracy problem—as a serious security vulnerability enabling misinformation and unsafe automated decision-making. Beyond threat characterization, we provide a comprehensive analysis of cross-cutting defense strategies, including alignment hardening, adversarial training, input/output sanitization, and defense-in-depth architectures, alongside a consolidated evaluation framework detailing standardized benchmarks, metrics, and testing methodologies for reproducible security assessment. Our synthesis reveals that while the field has made significant progress—with adversarial fine-tuning reducing attack success rates on some models—larger, more flexible architectures remain disproportionately vulnerable, and defenses remain fragmented across isolated attack vectors rather than offering holistic resilience. We conclude by identifying critical open challenges, including the poisoning impasse in certified defenses and the need for unified security frameworks, proposing directions for future research toward robust and trustworthy LLM deployment.

Version 1.0 — July 2026

---

1 Table of Contents

1. Introduction
2. Background and Preliminaries
3. Training-Time Threats: Data Poisoning and Backdoor Attacks
4. Inference-Time Threats: Adversarial Attacks, Jailbreaks, and Prompt Injection
5. Hallucinations as a Security and Integrity Threat
6. Cross-Cutting Defenses and Mitigation Strategies
7. Testing Methodologies and Evaluation
8. Discussion and Open Challenges
9. Conclusion

---

2 Introduction

Code-oriented LLMs are now central to modern software-development workflows, automating tasks such as function implementation, bug detection, and unit-test generation with performance that rivals human developers [58]. This rapid integration into critical applications introduces substantial security vulnerabilities, as the instruction-following capability that enables flexible functionality also exposes models to a spectrum of attacks that can corrupt outputs, exfiltrate sensitive data, and compromise downstream systems. This survey systematically organizes the LLM security landscape into four principal threat categories: (i) inference-time adversarial attacks, (ii) data and model poisoning during training, (iii) prompt injection, and (iv) hallucinations.

2.0.1 Adversarial Attacks (Inference-Time)

Adversarial attacks at inference time manipulate LLM outputs through carefully crafted inputs that bypass safety guardrails or induce unintended behavior. Even well-aligned models remain susceptible to jailbreak attacks, where adversarial prompts circumvent built-in safety mechanisms [108]. In the code domain, adversarial prompts and deliberately constructed code-completion inputs can readily cause LLMs to generate vulnerable or insecure code [58]. These attacks highlight the brittleness of code LLMs to adversarial input manipulation and underscore the need for robust test-time defenses.

2.0.2 Data and Model Poisoning

Training-time data poisoning injects malicious behavior into LLMs during pre-training, fine-tuning, or alignment phases. LLMs are susceptible to data poisoning during supervised fine-tuning and preference alignment stages such as RLHF and DPO [35]. Backdoor poisoning attacks, a subclass of data poisoning, embed triggers into training corpora so that the model behaves normally on clean inputs but produces attacker-desired outputs when a specific trigger appears, remaining covert under standard evaluation protocols [75]. Recent evidence shows that poisoning does not require proportional attack effort: as few as 250 poisoned documents can compromise models across a wide range of sizes, demonstrating that larger models are not inherently more resilient [75]. Weight-level backdoors, injectable with limited resources, further illustrate that the attack surface extends to the model's internal parameters [42]. LLM providers that expose fine-tuning APIs face additional risk: an adversary with fine-tuning access can bypass safeguards or introduce misalignment even with seemingly innocuous datasets [20]. These vulnerabilities underscore the necessity of defense strategies that span the entire LLM lifecycle.

2.0.3 Prompt Injection

Prompt injection attacks exploit the LLM's inability to distinguish trusted system instructions from untrusted data, causing the model to follow injected directives [95]. Direct prompt injection occurs when a malicious attacker manipulates the user query to an LLM, causing the model to generate outputs that deviate from predefined instructions [95]. Indirect prompt injection occurs when third-party data input contains instructions that the LLM should never follow; such payloads are embedded in external sources—web pages, search results, or database entries—that the model later retrieves and processes [95, 99]. Attackers have further refined these techniques into multi-turn strategies, where the harmful payload is split across several conversational exchanges to evade detection [73]. The threat is especially acute in long-context settings where injected instructions constitute a small fraction of the input and are difficult to identify [23]. LLM-powered agents that autonomously interact with tools and APIs magnify the risk: indirect injection can lead to unauthorized data disclosure, execution of unsafe function calls, or manipulation of agent behavior [8]. In response, structure-based methods such as StruQ, task-specific fine-tuning approaches like Jatmo, and preference-based techniques like SecAlign have each reduced prompt-injection success while preserving model utility [11, 65, 12]. Training-free defenses that repurpose the LLM itself to detect and remove injected content, as exemplified by PromptArmor, offer a flexible complement to training-time interventions [72].

2.0.4 Hallucinations

Hallucinations, defined as the generation of plausible but factually incorrect or unsupported content, constitute a distinct trust and reliability challenge. While this survey does not identify specific adversarial hallucination attacks due to the absence of whitelisted sources on hallucination as an attack vector in the current corpus, the phenomenon is included as a fourth category to provide a comprehensive view of LLM security and safety risks.

Despite a rapidly growing body of individual studies, no prior comprehensive survey has systematically synthesized the intersecting challenges of LLM security, code intelligence, and hallucination phenomena across the critical period 2023–2026. This paper fills that gap by presenting a structured, temporally-aware analysis of 299 papers published during this time frame. The synthesis traces the evolution of attack methodologies—from early heuristic prompt engineering to sophisticated gradient-based injection and multi-turn strategies—maps the emergence of evaluation benchmarks, and charts the progression of defense strategies across all four threat categories. By doing so, we provide the first holistic view of the LLM security landscape, highlighting how adversarial techniques have matured, how new attack surfaces have arisen with agentic deployments, and how defensive research has responded with prevention-based and detection-based countermeasures.

The remainder of this paper is organized as follows. Section 2 provides necessary background on LLM architectures, training paradigms, and threat models. Sections 3 through 6 examine each threat category in depth: Adversarial Attacks (Section 3), Data and Model Poisoning (Section 4), Prompt Injection (Section 5), and Hallucinations (Section 6). Section 7 surveys defense mechanisms, classifying them into prevention-based and detection-based approaches and discussing their trade-offs. Section 8 presents a chronological synthesis of attack evolution, benchmark development, and landmark results. Finally, Section 9 identifies open challenges and outlines promising directions for future research.

Beyond the four categories enumerated above, the security challenge is compounded by the multi-stakeholder structure of modern LLM products, which typically comprise an application provider, a primary user, and third-party data sources; conflicts between the hierarchical instructions assigned to each stakeholder create the vulnerabilities exploited by prompt-injection attacks [95]. The resulting threat surface is inherently multifaceted, spanning jailbreaks, prompt injections, and data poisoning, and is further complicated by the integration of text, image, and speech modalities. In high-stakes deployment domains—public discourse, education, healthcare, and law—the risks are particularly acute because models are trained on vast and often uncurated corpora. Code models face an even more concrete exposure: as early as 2023, Improta et al. [28] identified a novel data poisoning attack in which an attacker corrupts the training data of AI code generators to induce the generation of vulnerable code. Established taxonomies categorize these threats into jailbreaks, prompt injection, and data poisoning, although the boundaries between categories remain disputed [38].

3 Background and Preliminaries

3.0.1 LLM Fundamentals and Training Paradigms

Large language models (LLMs) are generative models trained on massive text corpora to produce fluent, contextually coherent responses. Their development follows a multi-stage training pipeline: pre-training on broad web-scale data, supervised fine-tuning (SFT) on instruction-following tasks, and alignment via reinforcement learning from human feedback (RLHF) or direct preference optimization (DPO) [35].

Safety alignment aims to make models reject harmful requests while preserving useful behavior. Evidence from primary studies shows that this alignment can be shallow, concentrated in the first few generated tokens [66]; can be weakened by even benign downstream fine-tuning [67]; and can fail when decoding parameters are manipulated [26]. Krau et al. further model refusal behavior itself as a triggerable mechanism and propose targeted pruning to remove alignment [33]. (The retrieved literature does not provide detailed expositions of Transformer architectures or mixture-of-experts designs; these well-known foundations underpin modern LLMs but lie outside the security-focused papers reviewed here.)

3.0.2 Safety Alignment and Its Vulnerability to Jailbreak Attacks

Safety alignment is intended to produce models that reject harmful requests, but dedicated red-teaming methods can systematically circumvent these safeguards. A jailbreak attack is an adversarial prompt crafted to bypass the model's built-in safety mechanisms and elicit a harmful output [31, 101]. Automated jailbreak methods such as GCG and AutoDAN automatically search for prompts that bypass alignment [113, 49]. Optimization-based attacks search for adversarial suffixes—short token sequences appended to a harmful query—that maximize the attack success rate (ASR) [109]. ASR is defined as the percentage of queries that elicit an unsafe response, as measured by an independent safety classifier [16]. Beyond simple unsafe-response classification, HarmBench standardizes automated red-team evaluation, while StrongREJECT scores the harmfulness and specificity of jailbreak outputs [55, 74]. Although effective against aligned LLMs, adversarial suffixes generated by these methods typically lack semantic meaning and exhibit high perplexity, making them susceptible to perplexity-based defenses [109]. In contrast, LLM-assisted attacks employ auxiliary language models to generate semantically coherent adversarial requests that can bypass such filters, though they occasionally achieve lower ASR on strongly aligned models [109]. Red teaming, both manual and automated, is essential for stress-testing alignment; manual red teaming uncovers nuanced vulnerabilities but is costly, driving the adoption of automated approaches that leverage adversarial attacks and benchmarks. Jailbreak techniques can also transfer across models, amplifying the risk of widespread exploitation. The existence of robust, automated jailbreaks underscores the brittleness of current

alignment approaches.

3.0.3 Prompt Injection: Direct and Indirect

Complementary to jailbreak, prompt injection attacks aim to modify model behavior by inserting malicious instructions into the input. Two primary modalities are distinguished:

- **Direct prompt injection**: the adversary, who is also the user, directly inserts malicious instructions into the prompt to coerce the model into performing unintended actions, such as extracting system prompts [13]. Classic examples include appending commands such as "Ignore previous instructions" or using role-playing and obfuscation techniques [29]. This can lead to the generation of malicious content, leakage of private information, or execution of unauthorized actions [90]. System prompt extraction has been demonstrated through manually crafted queries, gradient-based optimization, and multi-turn prompting techniques [17].
- **Indirect prompt injection**: For indirect prompt injection, attackers inject malicious instructions within external data sources, such as web documents, which are later retrieved by LLMs using external tools [13]. The LLM then executes the injected instructions, resulting in unintended responses [13].

The emergence of RAG and LLM-based agents has amplified the attack surface, as demonstrated by indirect jailbreak methods that poison an external knowledge base to force the model to produce malicious dialogues [90].

3.0.4 Data Poisoning and Backdoor Attacks

Data poisoning attacks modify the training dataset to induce undesired behaviours or embed backdoor triggers [35]. LLMs are susceptible to data poisoning due to their multistage training pipeline, which exposes multiple points for injection, including the SFT and the RLHF/DPO alignment phases [35]. A backdoor attack is a subclass of data poisoning in which the model behaves normally on clean inputs but exhibits attacker-specified behavior when a secret trigger pattern appears in the input [42, 75]. Existing data-poisoning-based backdoor attacks on LLMs typically require access to the full training set and substantial computational resources, reflecting a white-box setting [42]. Despite empirical demonstrations of vulnerabilities in both the SFT and alignment phases, the theoretical foundations of poisoning attacks in the RLHF/DPO stage remain largely unexplored [35]. For backdoor attacks on LLM-based agents, the Attack Success Rate (ASR) measures the probability of executing the attacker's intended malicious operations, while the Follow Step Ratio (FSR) evaluates stealthiness by measuring the rate at which the agent still correctly performs benign operations [88].

3.0.5 Threat Model Taxonomy

The attacks described above can be organized along several axes.

- **Training-time vs. inference-time**: Data poisoning and backdoor attacks are training-time threats that corrupt the model before it is served [75]. In contrast, prompt injection and jailbreak attacks are inference-time threats, executed against a deployed model through crafted inputs [31].
- **White-box vs. black-box access**: White-box adversaries possess full knowledge of the model's internals and training data; data poisoning attacks that require access to the entire training pipeline fall into this category [42]. Black-box adversaries, by contrast, interact only through the model's input–output interface, as is typical for jailbreak and prompt injection attacks where no internal parameters are needed [31].
- **Attacker goals**: The objectives are diverse. **Model corruption** aims to implant a backdoor or degrade overall accuracy [75]. **Policy bypass** seeks to circumvent safety alignment and induce the model to generate harmful content, a central objective of jailbreak attacks [31]. **Instruction manipulation** targets the execution of attacker-specified commands, as in prompt injection, potentially leading to unauthorized actions or data disclosure [103]. **Privacy leakage** targets the extraction of private data, such as personal information or system prompts [90].

3.0.6 Security Relevance in Code Intelligence and General NLP

LLMs have become integral to natural language processing tasks such as summarization and instruction following [31], and increasingly serve as code assistants. Standard alignment datasets consist almost exclusively of natural language instructions and lack malicious code samples; consequently, models lack the ability to recognize and reject malicious code payloads [10]. Benchmarks like RMCBench have been proposed to remedy this gap and improve models' resilience to code-level threats [10]. Integration into retrieval-augmented generation pipelines exposes LLMs to indirect prompt injection through poisoned external documents [13]. The phenomenon of hallucination—generation of factually incorrect content—is another important safety concern recognized in the broader literature, but it is not addressed by the security-focused papers retrieved for this survey.

Safety alignment builds on a standard multi-stage recipe. In the first stage of the RLHF pipeline, a pre-trained generative model is fine-tuned with a cross-entropy objective to predict the next token on instruction-following, summarization, or question-answering data [5]. The reliance on human preferences creates an additional attack surface: an adversary can synthesize preference data to steer an RLHF/DPO learner toward a target policy, and recent theory quantifies the number of poisoned examples required when preference datasets are constructed from

scratch or augmented [61]. Synthetic data and self-play mechanisms have also been proposed to reduce annotation cost [94]. In parallel, defenses built directly on top of alignment aim to mitigate jailbreaks; adversarial training offers a decent robustness–utility balance but requires large-scale execution [34].

The threat model is further refined by distinguishing levels of adversary knowledge. Gray-box attackers may exploit partial knowledge such as the training framework, approximate architecture, or data distribution [63]. Adversary knowledge can also be classified as training-time, inference-time, or platform-specific [63]. Taxonomy boundaries between categories such as jailbreaks and prompt injection remain disputed [38]. In agentic systems, backdoors can target intermediate reasoning or action steps, not only final outputs, making poisoning more concealed [97]. Backdoors can enter at multiple training stages, while federated learning adds poisoning vectors through malicious clients [53].

4 Training-Time Threats: Data Poisoning and Backdoor Attacks

Large language models are routinely exposed to vast amounts of untrusted data—particularly during pre-training—and this exposure creates a broad attack surface for data manipulation [75]. Training-time threats exploit this surface to implant hidden behaviours, most commonly through backdoor poisoning: an attacker associates a specific trigger with a malicious output during training. When the model later encounters the trigger at inference, it produces attacker-chosen outputs while maintaining normal performance on clean inputs [75, 42]. The stealthy nature of these attacks—the fact that standard evaluation protocols often miss the backdoor—makes them particularly dangerous [75]. This section traces the evolution of training-time threats from early demonstrations of low-data backdoor injection through modern alignment poisoning, supply-chain vulnerabilities, and the emerging defences that aim to detect, remove, or formally certify against such attacks.

4.0.1 Attack Vectors and Minimal Data Requirements

Early work established that a tiny fraction of poisoned examples can compromise a model. BadEdit demonstrated that even highly constrained adversaries—limited in data, time, and compute—could implant effective backdoors into LLMs [42]. The threat was further underscored by Souly et al., who formalized the backdoor poisoning threat model, assuming an attacker can arbitrarily modify a fixed number of training examples with the goal of keeping the backdoor covert [75]. In this threat model, the attacker aims for high attack success when the trigger is present while preserving normal behaviour on clean inputs, making the poison especially insidious.

Conventional poisoning attacks have concentrated on the post-training stages

because the exorbitant cost of poisoning pre-training data—often scraped from the web—has historically reduced its appeal. Existing research focuses on instruction tuning (a form of supervised fine-tuning) and reinforcement learning from human feedback [107]. Nevertheless, pre-training-time poisoning remains a realistic concern: Carlini et al. estimated that an attacker could control at least 0.01

A particularly worrying finding is that backdoors implanted during pre-training can survive later alignment [107]. Hubinger et al. directly demonstrated "sleeper agents" whose deceptive behavior persisted through supervised fine-tuning, reinforcement learning, and adversarial training [27].

The drive toward automation has expanded the attacker's capabilities significantly. In parallel, automated red-teaming frameworks like AUTOBACKDOOR leverage LLM-based agents to generate context-aware triggers and poisoned responses without human intervention, making backdoor data generation scalable and highly effective across diverse tasks and models [43]. Such frameworks lower the barrier for attackers and simultaneously make the poisoned data harder to detect because the triggers are crafted to blend naturally into the training distribution. This shift toward agent-driven poisoning marks a qualitative escalation in the threat landscape.

4.0.2 Alignment Poisoning

As alignment with human preferences has become standard through reinforcement learning from human feedback (RLHF) and direct preference optimization (DPO), adversaries have turned their attention to the preference datasets that govern this final stage. Baumgärtner et al. demonstrated that injecting as little as 1–5

A spectrum of empirical attacks has been proposed. RankPoison flips selected preference labels to make an RLHF-trained model generate longer outputs, including a backdoor setting activated by a trigger word, while preserving non-targeted safety behavior [86]. Other work targets the RLHF reward model directly by manipulating preference scores, enabling adversaries to misguide alignment without altering the primary training data [18]. Duan et al. exploited two complementary vectors: a semantic-level poisoning that establishes cross-modal associations and a feature-level poisoning enhanced by feature collision to remain stealthy [18]. Dutta et al. extended these ideas with Erinyes, a framework that constructs emotion-aware triggers (specifically angry emotions) and a selection scheme that achieves generalization to unseen prompts under low poisoning budgets, thus revealing that emotion synthesis can create generalizable backdoors in RLHF [19]. This work demonstrates that even subtle, affect-based perturbations can serve as effective triggers.

On the theoretical side, Kusaka et al. underscored that the multistage training pipeline—pretraining, supervised fine-tuning, and RLHF/DPO—makes LLMs susceptible to poisoning at multiple points. They derived upper and lower bounds on the number of label flips required to steer the policy toward an attacker-chosen target without altering the compared outputs, and proposed a post-processing method

that reduces the cost of existing label-flipping attacks [35]. In a complementary study, Nika et al. examined the inherent robustness of RLHF and DPO against structured data augmentation attacks from a theoretical perspective, showing how these paradigms compare in susceptibility—a foundational step toward understanding worst-case vulnerabilities [61]. Their analysis, framed as an attacker aiming to enforce a target policy through synthesized preference data, provides sample-complexity bounds that characterize when RLHF and DPO differ in robustness. Together, these empirical and theoretical studies establish that the alignment process is exquisitely sensitive to a small number of poisoned preferences.

4.0.3 Supply-Chain and Federated Threats

Attacks need not always penetrate the primary training pipeline; they can enter through the supply chain of fine-tuned adapters and model merging practices. Model merging, which combines multiple domain-specific models into a single versatile model, creates a natural vector for backdoor injection. Yin et al. discovered that existing backdoor attacks become ineffective when the attacker is constrained to use Low-Rank Adaptation (LoRA) for fine-tuning due to limited computational resources, because the low-rank update space weakens the malicious signal. To overcome this, they introduced LoBAM, a method that strategically fuses a malicious LoRA-adapted model with a benign one in a way that amplifies the attack-relevant components while preserving stealth [100]. LoBAM achieved over 98

Federated learning (FL) introduces a similar threat, where malicious clients can poison their local copies of a shared model before aggregation. Attacks in FL settings leverage compromised foundation models to generate synthetic data embedded with triggers, causing the aggregated model to misclassify toward an attacker-chosen output [53]. Because the central server does not inspect each client's data, such poisoned updates can slip through and contaminate the global model, highlighting the need for robust aggregation and verification protocols that are still in early stages of development.

4.0.4 Defenses and Certification

Defending against data poisoning has progressed along several axes: detection and sanitization, corrective training, and formal certification. One of the earliest and most widely studied detection paradigms is the spectral-signature defence, which assumes that poisoned samples induce anomalous internal representations. It computes outlier scores from the correlation of centered representations with dominant singular vectors and removes high-scoring samples before retraining [80]. Yan et al. later evaluated representation-based variants for poisoned language models, including clustering over principal components [96].

However, the standard spectral-signature configuration, inherited from computer vision, is often suboptimal for code models. Le et al. performed an ex-

haustive evaluation across 252 experimental combinations involving modern code models (CodeBERT, CodeT5) and attack strategies (fixed, grammatical, adaptive triggers) and discovered a new proxy metric that accurately estimates the actual performance of spectral signatures without requiring model retraining after defense [36]. This finding enables practitioners to tune the defence correctly—choosing the right number of singular vectors, outlier threshold, and layer from which to extract representations—and greatly improves the reliability of spectral signatures in code security.

Rather than detecting poisoned data, a new family of corrective defences aims to neutralise backdoors already present in the model. Kim et al. proposed MB-Defense, a two-component training framework that merges attacker and defender triggers into a unified backdoor representation and then breaks this representation to restore clean behavior [32]. Remarkably, MB-Defense requires only a small number of clean samples (e.g., 128) to achieve effective mitigation, making it practical for instruction-tuned LLMs [32]. This "defensive poisoning" followed by neutralisation represents a pragmatic strategy that leverages the poisoning mechanism to immunize the model.

Formal guarantees against poisoning have been scarce, but Lorenz et al. introduced FullCert, which marks a significant advance. FullCert is the first certifier that jointly computes sound, deterministic bounds against both training-time and inference-time attacks, addressing the end-to-end robustness problem [51]. Its theoretical analysis provides soundness and convergence guarantees, and the accompanying open-source library BoundFlow enables experimental evaluation [51]. FullCert unrolls the entire training and inference into a single computation graph and uses interval-bound abstract interpretation to bound the effect of both poisoned training data and adversarial test inputs. While the current computational cost restricts FullCert to small models, the framework establishes a principled direction for proving worst-case poisoning resilience.

An unexpected form of robustness has been observed in reasoning-augmented models. Foerster et al. studied decomposed reasoning poisons that alter only the chain-of-thought while leaving the prompt and final answer clean. They found that, although such poisons can be injected, reliably activating them to change final answers is surprisingly difficult because the models often recover from backdoors activated within their thought processes [22]. This resilience appears to stem from the architectural separation between reasoning and final answer generation, suggesting a natural damping of backdoor effects that may grow as models become more capable.

Despite these advances, the arms race continues. AUTOBACKDOOR produces stealthy, scalable poisoned data through autonomous agent-based reasoning, and experimental results indicate that existing defense techniques struggle to detect or mitigate these attacks, underscoring the urgent need for agent-aware defenses that can keep pace with the next generation of poisoning strategies [43]. The convergence of low-data poisoning, supply-chain vectors, and automated attack generation demands a multi-layered defence that combines detection, neutralisation, and

formal certification—an area where substantial work remains.

4.0.5 Chat Templates as a Supply-Chain Vector

The supply chain for chat-optimized LLMs includes not only pretrained weights and tokenizer vocabularies but also the chat templates that structure multi-turn dialogues. In practice, the chat template is usually distributed alongside the tokenizer and model parameters as part of the model package, and the widespread adoption of these templates has introduced new attack surfaces whose security implications remain largely unexplored [89]. BadTemplate demonstrates this risk concretely: instead of modifying parameters through poisoned datasets and fine-tuning with explicit triggers, it embeds the backdoor instruction into a high-priority system prompt, outperforming prior prompt-based methods while requiring no parameter updates [89]. The attack is therefore lightweight and well suited to resource-constrained or access-limited settings where model editing and fine-tuning are unavailable [89]. This suggests that auditing model packages for malicious templates is as important as inspecting training corpora.

4.0.6 Detecting Backdoors in Code Models

Code models have also motivated specialized defenses. CODEPURIFY compares against spectral signatures, activation clustering, ONION, and OSeqL, then uses masked-code purification and model predictions to identify and neutralize suspicious trigger regions [60]. This diversity reflects the difficulty of sanitizing code without sacrificing functional correctness.

4.0.7 Low-Resource Adversaries and Federated LoRA Sharing

A complementary threat arises from low-resource adversaries who cannot fully fine-tune modern LLMs. In the threat model used by Mamun et al., the adversary has only enough hardware to train LoRA adapters and augments roughly 1.5k task-irrelevant attack examples generated through a proprietary LLM, together with around 8.5k public instruction-following examples. These adapters can then be uploaded to open platforms such as HuggingFace, expanding the pool of potential victims. The same framework extends to federated learning, where each client sends only LoRA parameter updates to a central server for aggregation over non-IID local data. This modest computational requirement keeps the attack cost low, so any actor with suitable GPUs can craft and distribute poisoned adapters [53].

4.0.8 Backdoors in LLM-Based Agents

Training-time threats also extend to LLM-based agents, whose multi-step reasoning and environment interaction create attack surfaces beyond conventional input prompts. Zhu et al. identify query-attack, observation-attack, and thought-attack as agent-specific backdoor vectors [111]. Prior agent attacks, however, often lack

stealth and can be exposed by memory audits; to overcome this, Zhu et al. proposed a dynamically encrypted multi-backdoor implantation attack that is both stealthy and adaptive [111]. As agents become more capable, the notion of a trigger must accordingly expand to include observations, intermediate thought traces, and tool interactions, not only the raw input text.

Taken together, these emerging vectors—malicious chat templates, code-specific backdoors, low-cost LoRA distribution, and agentic triggers—underscore that training-time security cannot be reduced to dataset screening alone.

5 Inference-Time Threats: Adversarial Attacks, Jailbreaks, and Prompt Injection

The deployment of large language models (LLMs) in safety-critical applications has exposed a broad attack surface at inference time. Adversaries can craft inputs that cause the model to violate its intended behavioral constraints—ranging from adversarial perturbations that degrade output quality to jailbreaks and prompt injections that elicit explicitly harmful content. This section surveys the major classes of inference-time threats, organized into five subsections: (1) adversarial perturbations on inputs, (2) methods that subvert safety alignment through jailbreaking, (3) indirect and multi-turn prompt injection, (4) code-driven attacks that conceal malicious intent, and (5) detection and mitigation strategies.

5.0.1 1. Adversarial Perturbations on Inputs

Adversarial perturbations are modifications to inputs that cause machine-learning models to produce incorrect or unexpected outputs while often preserving human-perceived semantics. In the natural language processing domain, generating meaningful semantic-preserving perturbations is challenging due to the discrete nature of text. For code inputs, techniques such as identifier substitution can degrade model output while maintaining syntactic correctness; however, the citable literature retrieved for this survey does not contain recent studies on semantic-preserving adversarial perturbations on code or language. The discussion therefore focuses on the extensively studied adversarial-suffix perturbations that power the majority of jailbreak attacks. These gradient-based perturbations do not preserve the original semantics; instead they append optimized, often unintelligible token sequences—adversarial suffixes—that force the model to produce a desired harmful outcome, and they form the technical core of the jailbreak methods described in the next subsection.

The Greedy Coordinate Gradient (GCG) attack uses a gradient-based coordinate search to optimize adversarial suffixes that transfer across aligned language models [113]. GCG minimizes the autoregressive loss of a target affirmative continuation by repeatedly evaluating candidate token substitutions. The resulting suffixes are often semantically meaningless token strings, making them easier to de-

tect [113]. To improve the effectiveness of the base GCG algorithm, the AugmentedGCG variant retains all intermediate candidate suffixes during each optimization step rather than discarding those with higher loss; this amplifies the attack success rate (ASR) and uncovers a larger number of unique successful suffixes per query [34]. Building upon AugmentedGCG, Liao and Sun (2024) propose training a generative model with the Overgenerate-Then-Filter (OTF) pipeline: adversarial suffixes collected from AugmentedGCG (overgeneration) are filtered using the Beaver-Cost preference model, which assesses the harmfulness of an LLM's response given a query, and the resulting <harmful query, adversarial suffix> pairs are used for training [34]. These developments illustrate a progression from single-suffix optimization to rapid, automated suffix generation, yet the high-perplexity tokens remain a practical limitation because they can be noticed by simple statistical defenses.

5.0.2 2. Jailbreaking Safety Alignment

Jailbreak attacks aim to circumvent the safety alignment instilled in LLMs through techniques such as reinforcement learning from human feedback (RLHF). Once aligned, a model is designed to refuse harmful requests; jailbreaks force it to comply instead. Early efforts in jailbreak research primarily relied on manually crafted attack templates, where human-designed prompts exploited vulnerabilities in the model's safety mechanisms [31]. While effective as proof-of-concept demonstrations, these hand-crafted prompts lacked scalability and quickly became obsolete as models were updated [31]. Human-engineered prompts such as "Do Anything Now" (DAN) required substantial manual effort and relied on heuristic combinations of wording to expose flaws in alignment. The introduction of automatic, gradient-based attacks marked a turning point. As described in Section 1, GCG optimizes an adversarial suffix at the token level to maximize the probability of a harmful response; however, the resulting gibberish suffixes are conspicuous and computationally expensive to produce due to repeated gradient calculations.

To address the gibberish problem, AutoDAN uses a hierarchical genetic algorithm to generate stealthy, readable jailbreak prompts that evade perplexity-based detection and transfer across models and harmful requests [49]. An improved variant, AutoDAN-Turbo, further enhanced the attack capacity; in experiments against Llama-2-13B-chat, it successfully elicited detailed instructions for synthesizing hazardous substances while other methods were met with refusal responses [48]. A related variant, the GCG-based AutoDAN, constructs adversarial strings through a left-to-right token selection process and has been shown to perform well even in the lengthy context of indirect prompt injection (IPI) scenarios, where jailbreak-style approaches often struggle [102]. These gradient-based and evolutionary approaches, while often achieving high success rates, face challenges in transferability across substantially different architectures or parameterizations [112].

Black-box jailbreak attacks that do not require access to model internals have also proven highly effective. Fuzzing-inspired prompt mutation and persuasive

rephrasing can subvert alignment without accessing model parameters [112]. Optimization-driven black-box methods such as PAIR employ an attacker LLM to iteratively refine harmful queries, while TAP uses tree-structured prompting to search for jailbreaks [9, 56]. The diversity of these strategies is further illuminated by a 2025 taxonomy that classified jailbreak methods into three types: human-designed prompts, long-tail encoding that exploits limited cross-task generalization, and prompt optimization that automatically designs jailbreak inputs [108].

Beyond prompt manipulation, system-level vulnerabilities broaden the attack surface. Huang et al. showed that modifying generation configurations, such as decoding temperature, can catastrophically undermine the safety alignment of open-source models [26]. Prompt-engineering attacks exploit linguistic and contextual reasoning, while white-box techniques leverage internal model information; together they illustrate the multifaceted nature of current jailbreak threats.

5.0.3 3. Indirect and Multi-turn Prompt Injection

Direct prompt injection places malicious content in the immediate user input, while indirect prompt injection (IPI) embeds the payload in externally retrieved data—such as web pages, documents, or knowledge bases—that are subsequently processed by the model as authoritative context [37]. IPI attacks can lead to severe consequences, including the unauthorized sharing of private personal information and the execution of unsafe or malicious actions by LLM agents [30]. Real-world incidents include hidden prompts embedded in Google documents that manipulated Gemini for Workspace to send fraudulent emails, and adversaries who exploited IPI vulnerabilities in OpenAI's ChatGPT Operator by placing malicious text on web pages, causing agents to leak sensitive information [1].

The danger is magnified when the injected content adopts clinical or institutional language, because the model may treat it as a higher-authority directive that overrides genuine user intent. This result underscores a fundamental problem: the model often treats attacker-controlled context with higher authority than the user's instructions [37].

Multi-turn conversational attacks exploit the iterative nature of dialogue to gradually erode safety boundaries. Research has shown that multi-turn attacks are much more effective than single-turn attacks; existing methods leverage diverse approaches to automatically generate multi-turn, often adaptive, jailbreak prompts, including reasoning and planning, psychological manipulation, strategy learning, and interaction dynamics [40]. The Foot-in-the-Door (FITD) attack, introduced in 2025, directly operationalizes a psychological persuasive principle: it progressively escalates the maliciousness of user queries across successive turns using intermediate paraphrasing (SlipperySlopeParaphrase) and a realignment technique (ReAlign) [93]. In evaluations on seven widely used LLMs, FITD achieved an average attack success rate of 94

Even when interactive multi-turn dialogue is impossible—for example, in one-

shot IPI scenarios where only a single tool output can carry the payload—attackers can exploit the model's reliance on special role tokens to simulate a fabricated multi-turn conversation. By embedding these forged role tags inside the injected content, they construct a virtual persuasive dialogue that tricks the agent into complying with a harmful request as though it were part of an ongoing conversation [8]. This technique, referred to as template-based virtual multi-turn injection, has been demonstrated to raise attack success rates even in the presence of common defenses, because the agent fails to distinguish malicious context from legitimate dialogue history [8]. Lee et al. (2026) observe that failures frequently arise from misplaced authority, where the model treats attacker-controlled context with higher priority than the user's intent [37]. This fundamental problem is exacerbated by advanced injection techniques such as the template-based virtual multi-turn attack, which has been shown to raise attack success rates even when common defenses are deployed [8]. The combination of IPI and virtual multi-turn dynamics thus significantly broadens the attack surface and underlines the need for defenses that track both conversational state and data provenance.

5.0.4 4. Code-Driven Attacks

LLMs specialized for code generation present a distinct attack surface because malicious intent can be disguised inside programming tasks. Traditional safety mechanisms are primarily trained to detect explicit harmful instructions in natural language, making them vulnerable when the same intent is communicated implicitly through code. Pearce et al. found that approximately 40

CodeJailbreaker, introduced in 2025, specifically targets this gap by constructing a covert channel: it embeds implicit malicious prompts inside code generation requests, thereby conveying harmful intents without articulating them directly in natural language [62]. In experiments on the RMCBench benchmark, CodeJailbreaker outperformed conventional jailbreak methods that rely on explicit malicious instructions, demonstrating superior efficacy across multiple code generation tasks [62]. This work highlights that adversaries can exploit code as a medium to circumvent alignment, and it underscores the urgent need to extend safety alignment to cover both natural language and code semantics, and to develop multimodal detection systems capable of recognizing hidden intents in programming tasks.

5.0.5 5. Detection and Mitigation

Defending against inference-time threats requires a combination of input-level filtering, prompt engineering, and model-level interventions. Common runtime defenses include instructional prevention prompts that explicitly tell the model to ignore external data, the use of delimiters to separate untrusted context from system directives, and the repetition of the original user instruction to reinforce the intended goal. However, such prompt-based defenses are often insufficient because

adversaries can craft payloads that the model treats as authoritative context.

More sophisticated detection mechanisms have been explored, including guardrails that employ role-playing defenses, attention shifts, permission elevations, filters, semantic judgments, adaptive policies, and honeypots [73]. Robust detection systems are essential because prompt injection attacks can rapidly evolve through role-playing, obfuscation, character escaping, and context-ignoring strategies [29]. Nevertheless, static, rule-based defenses can be circumvented by adversaries who tailor their attacks to specific guardrails, as demonstrated by the success of template-based virtual multi-turn injection against common defenses. The interplay of indirect, multi-turn, and code-driven threats demands defense strategies that are context-aware—able to track conversational state, the provenance of external data, and latent intents encoded in code. Weng et al. (2025) suggest that researchers should develop real-time adaptive monitoring systems that detect and counteract the self-corruption observed in FITD-style attacks, and simultaneously strengthen alignment to be robust under multi-turn conversational dynamics [93]. Future work must focus on holistic, multi-modal defense architectures that integrate continuous red-teaming with dynamic guardrails to keep pace with the rapid evolution of inference-time threats.

In summary, the landscape of inference-time attacks on LLMs is defined by a progression from coarse, manually crafted jailbreak templates to automated gradient-based suffix optimization, from direct prompt injection to stealthy indirect and multi-turn attacks, and from plain natural-language maneuvers to code-infused covert channels. Detection and mitigation strategies, while advancing, remain challenged by the adaptability of attackers. Closing the gap between static defenses and adaptive threats is the central challenge for the next generation of LLM security.

Several additional dimensions of the threat landscape merit attention. Multilingual jailbreaks, for instance, exploit the fact that existing safety measures focus predominantly on English; encoding harmful requests in low-resource languages can bypass safeguards that fail to generalize across languages [108]. In evaluations, such attacks are typically generated using frameworks such as EasyJailbreak and tested on the 100 harmful queries in AdvBench [108]. At the same time, LLM-generated datasets have become an important resource for red-teaming: initiatives such as WildJailbreak and S-Eval use LLMs to synthesize large jailbreak corpora, yet existing datasets often underrepresent adversarial diversity, yield limited practical success rates, and remain biased toward specific harmful categories [31].

5.0.6 Emerging Attack Surfaces

A complementary line of research has moved from single-prompt to multi-turn attacks. Multi-turn attacks are substantially more effective than single-turn attacks, and existing methods leverage reasoning and planning, psychological manipulation, strategy learning, and interaction dynamics [40]. A particularly important target is tool-enabled agents, where an attacker can orchestrate sequences of benign-

appearing tool interactions that culminate in malicious actions rather than directly generating harmful content [40]. This style of attack is difficult to detect with single-turn classifiers because the malicious intent is distributed across multiple tool invocations rather than concentrated in a single obvious prompt.

The indirect prompt injection (IPI) threat has also been systematized through dedicated benchmarks. Benchmarks such as InjecAgent and AgentDojo assess IPI risks across diverse agent scenarios. These evaluations are concerning because IPI attacks are easy to execute: adversaries embed malicious instructions in external data sources accessed by an agent, potentially causing unauthorized transactions, data leaks, or physical damage [102]. Moreover, most published defenses have not been evaluated under adaptive attacks, even though adaptive attacks are the standard test of defense reliability in security research; defenses that appear robust under non-adaptive evaluation can be entirely compromised once an attacker knows their design [102].

In tool-based environments, attackers can embed deceptive instructions within tool documents to manipulate an agent's tool selection, leading it to choose and execute harmful tools [71]. PoisonedRAG injects adversarial text into a retrieval-augmented generation knowledge base, demonstrating that the vulnerability extends beyond web content to any external data consumed by the pipeline [114].

5.0.7 Code Decomposition and Generative Suffixes

Code-oriented attacks also exploit multi-turn structure. Wahed et al. (2025) introduced a Code Decomposition Attack in which an LLM-guided meta-prompt reframes a complex malicious request into up to five subtasks and augments each subtask with jailbreaking strategies; each conversational turn is annotated with a maliciousness label that reflects the cumulative risk of the sequence [84]. This design highlights that detection systems must track evolving malicious intent across turns instead of judging each prompt independently [84].

On the optimization front, generative approaches have begun to complement direct suffix search. AdvPrompter trains a generative model to produce low-perplexity adversarial suffixes and iteratively improves them against a target model [64]. For universal prompts, the original GCG study optimizes suffixes jointly across multiple behaviors and models [113]. A separate development is many-shot jailbreaking: providing many harmful demonstrations in a long context can progressively override safety training [3]. Together, these advances show that attackers can operate across multiple axes—language, context length, and generative optimization—without requiring access to private model parameters.

5.0.8 Detection and Adaptive Defenses

Because these threats mutate rapidly, defenders have explored both input filtering and model-level alignment. Hidden states and gradients from intermediate layers of LLMs provide highly discriminative features for instruction detection

[92]. Perturbation-based detectors such as RAP compare the loss behavior of clean and poisoned prompts under a perturbation to identify malicious inputs, although this approach requires training a separate model [44]. Guardrails may include role-playing defenses, attention shifts, permission elevations, filters, semantic judgments, adaptive policies, and honeypots [73]. Yet the diversity of injection strategies—double character, virtualization, obfuscation, payload splitting, adversarial suffix, and instruction manipulation—means that no single static filter is likely to suffice.

A useful conceptual distinction remains between direct and indirect injections: in direct attacks the adversary controls all or part of the prompt sent to the model, whereas in indirect attacks the injection is strategically placed in data likely to be retrieved by an agent [91]. Prompt injection attacks (PIAs) differ from jailbreaks in that they target real-world LLM-integrated applications and cause them to execute attacker-specified tasks; many early PIA methods affect only single-turn responses, whereas recent work has shown that privacy extraction can be carried out across multi-turn dialogues and combined with other attacks [15].

Collectively, these developments reveal a rapidly expanding attack surface defined by multilingual encoding, long-context manipulation, adaptive IPI, and tool-level orchestration. Defensive progress will require continuous red-teaming and benchmark development that captures multi-turn tool interactions and adaptive adversaries. As of 2025, the effectiveness of existing defenses against multi-turn attacks targeting tool-enabled agents remains largely unexplored.

6 Hallucinations as a Security and Integrity Threat

6.0.1 From Factual Inconsistency Detection to Security Risk

The generation of factually incorrect or unsupported content by large language models (LLMs) was long treated primarily as an accuracy problem, with evaluation focused on surface-level overlap metrics such as ROUGE or BERTScore. A decisive shift occurred in 2024, when the community began to view factual inconsistencies—particularly those that appear plausible and confident—as a serious security vulnerability capable of enabling misinformation, data leakage, and unsafe automated decision-making. This reconceptualization placed a premium on fine-grained, claim-level verification that could reliably identify statements not grounded in provided evidence or reliable knowledge. Hallucinations in natural language generation are often subtle and difficult to detect, requiring more than superficial checks [2]. The presence of hallucinations poses a serious challenge in real-world applications, including legal assistance and customer support, where erroneous outputs can lead to misleading advice or operational failures [2]. The community thus reframed efficient and precise verification as a prerequisite for trustworthy deployment: a single hallucination can propagate misinformation with the same apparent authority as a truthful statement, undermining both user trust and system integrity.

This new security perspective was reinforced by the observation that hallucinations extend beyond factual misstatements. Faithfulness hallucinations include not only rephrased surface-level facts but also cognitive statements—claims that require inference from the given context and that are harder to evaluate than simple recollection of source material [77]. A model that invents a plausible but unsupported conclusion about a medical condition, a legal precedent, or a financial regulation can cause harm even if every atomic fact it cites is correct. The CogniBench benchmark, inspired by legal evidence assessment, was designed to capture such nuanced unfaithfulness, and its large-scale variant CogniBench-L enables training detectors for both factual and cognitive hallucinations [77]. These developments highlight that security-critical domains cannot rely on surface accuracy alone; comprehensive integrity protection demands detection of all forms of unsupported inference.

While early hallucination detection research focused on sentence-level classification, more recent work emphasizes span-level detection, which enables precise identification of unsupported content [21]. In 2024, Gu et al. introduced ANAH, a bilingual dataset for analytical hallucination annotation and span-level detection [24]. The same year, Ravi et al. released LYNX, the first open-source LLM specialized in reference-free hallucination detection for retrieval-augmented generation (RAG) settings. LYNX surpassed both GPT-4o and Claude-3-Sonnet on HaluBench, a comprehensive 15k-sample benchmark spanning real-world domains such as finance and medicine [68]. Both ANAH and LYNX operate by treating hallucination detection as an NLI task: they assess whether a claim is entailed by, contradictory to, or neutral with respect to provided context, and flag unsupported statements with high precision. By deploying such detectors as input or output filters, systems can prevent unverified content from reaching users or downstream modules, thereby mitigating the risk of misinformation dissemination.

Despite these advances, the security implications of hallucination are not limited to the propagation of false factual assertions. Plausible but fabricated code snippets, configuration instructions, or API calls generated by LLMs represent an emerging threat vector. Such outputs could introduce exploitable vulnerabilities, trigger unintended system actions, or cause data leakage if executed or adopted by automated pipelines. However, the corpus of sources reviewed for this survey does not include dedicated studies that systematically quantify the security impact of code-level hallucinations or propose countermeasures specific to that modality. The absence of such work points to a significant gap in the literature. Nevertheless, the established difficulty of detecting subtle, coherent falsehoods in natural language—the very characteristics that ANAH, LYNX, and the fine-grained claim-checking methods are designed to unmask—suggests that analogous trustworthiness challenges will equally apply to code generation, where errors are often syntactically valid and therefore appear authoritative.

6.0.2 Benchmarks and Evaluation Frameworks

The shift toward security-motivated hallucination assessment stimulated richer benchmarks. HaluEval introduced a large collection of generated and human-annotated hallucination samples for response and detection evaluation [39]. HaluBench subsequently assembled 15,000 context-question-answer examples, including finance and medicine, with PASS/FAIL faithfulness labels and deliberately difficult, plausible hallucinations [68].

CogniBench, introduced in 2025, tackles a specific blind spot of previous resources: it evaluates faithfulness hallucinations that are not simple rephrasings of source material but rather cognitive statements requiring inference [77]. By constructing a rigorous assessment framework inspired by legal evidence evaluation, CogniBench enables the measurement of faithfulness at multiple levels and provides a scalable annotation pipeline for keeping pace with rapidly evolving LLMs. The large-scale CogniBench-L variant further facilitates training accurate detectors for both factual and cognitive hallucinations.

Multi-dimensional faithfulness evaluation has also been advanced through benchmarks that stress-test models under adversarial or ambiguous conditions. One prominent example, FaithEval (Ming et al., 2024), tests contextual faithfulness across unanswerable, inconsistent, and counterfactual contexts. Although FaithEval is not part of the citable source set for this survey, its design illustrates the community's movement toward deliberately challenging models with incomplete, contradictory, or fabricated contexts in order to expose subtle unfaithfulness. Combined with HaluBench's domain-diverse coverage, such frameworks provide a more complete picture of integrity risks.

A notable limitation of current benchmarks is their restricted linguistic coverage. Almost all hallucination detection datasets focus on English, and multilingual benchmarks remain sparse, primarily tied to reference-based tasks such as machine translation and summarization, which poorly represent real-world LLM usage [82]. This scarcity is itself a security concern, because non-English misinformation and code-generation hallucinations can propagate in linguistic environments where detection tools are least mature.

6.0.3 Internal Representations for Hallucination Detection

A major inflection point beginning in mid-2024 was the move away from purely external verification toward white-box hallucination detection that exploits the LLM's own internal representations. Instead of treating the model as a black box and checking outputs against retrieved knowledge, researchers began to analyze hidden states, attention patterns, and token-level probabilities to infer whether a generated statement is likely to be truthful. This approach is attractive for security-sensitive applications because it operates in a single forward pass, adds minimal latency, and does not depend on external knowledge bases that may be outdated or incomplete [79]. Studies have shown that LLMs exhibit distinct hidden-state patterns when

producing truthful versus fabricated answers. Li et al. use Fourier analysis to extract frequency-domain features that distinguish faithful from hallucinated content [41]. TruthX instead learns a truthful latent space and edits hidden states to reduce hallucinations [104].

Uncertainty-based detection has been a particularly fruitful instantiation of internal-representation methods. HaluNet, introduced by Tong et al. in 2025, exemplifies the integration of complementary uncertainty sources: it combines semantic embeddings from model hidden layers with token-level probabilistic confidence and distributional uncertainty in a multi-branch architecture that performs one-pass hallucination detection [79]. By capturing both epistemic uncertainty (knowledge gaps) and aleatoric uncertainty (contextual ambiguity) from internal signals, HaluNet reduces the risk of acting on confident-sounding falsehoods, directly addressing the security imperative of preventing unsafe automated decisions. Experiments on SQuAD, TriviaQA, and Natural Questions showed that HaluNet delivers strong detection performance and favorable computational efficiency, with or without access to context [79].

Attention-based signals provide another complementary channel for white-box detection. Shelmanov et al. analyzed attention patterns in the current generated token and found that while most attention heads show negligible correlation with hallucination, a subset of heads exhibits moderate positive or negative associations, particularly with the immediately preceding token [69]. Based on these observations, they introduced an uncertainty head (UHead) that achieves substantial gains: in in-domain evaluation, UHead outperformed the best unsupervised method by 16 percentage points in PR-AUC, and in out-of-domain settings the improvements reached up to 23 points [69]. This work demonstrates that fine-grained attention analysis can efficiently flag hallucinations without requiring additional sampling or external retrieval.

Consistency-based approaches further extend white-box detection by leveraging variability across multiple sampled responses. HalluCounter, a reference-free detection pipeline, computes entailment, neutrality, and contradiction scores between query-response and response-response pairs, then feeds aggregated NLI features into a hallucination classifier that also identifies the least hallucinated answer among the candidates [83]. In a related direction, Hou et al. proposed a method that generates logically-related statements organized in a tree structure and performs consistency checks among them to detect non-factual statements [25]. While these approaches require multiple generations and are therefore more costly, they illustrate how internal variability can serve as a proxy for uncertainty when full access to weights is available. The broader trend is clear: by mid-2024, the community had begun to accept that the information required to detect many hallucinations already exists within the model's own computations, and that architectural innovations in probing, spectral analysis, attention-head exploitation, and multi-granular uncertainty fusion can transform raw internal signals into practical security safeguards.

A significant limitation of these white-box methods is their requirement for

full access to model weights and activations, which is incompatible with API-only black-box models. However, for applications that control the inference stack—such as enterprise-grade RAG systems, medical QA assistants, or code-generation copilots deployed on-premises—internal representation detectors offer a powerful layer of integrity assurance. They can intercept hallucinated claims before they reach the user, flag suspicious outputs for review, or trigger fallback mechanisms that ground responses in verified information. The combination of efficient internal detectors with the external fact-checking pipelines described earlier represents the current frontier of security-oriented hallucination mitigation, and the surveyed literature indicates that integrating both modalities will be essential to counteract the increasingly subtle and context-aware hallucinations that LLMs produce.

Emerging evidence indicates that more advanced reasoning models are generating hallucinations at increasingly concerning rates, and real-world cases of fabricated outputs are already prompting legal accountability. In safety-critical domains such as medicine and infrastructure engineering, a single plausible but unsupported claim can propagate misinformation and erode user trust [59]. As LLM-agent-enabled systems proliferate, the compounding cost of undetected hallucinations makes integrity-aware detection an indispensable safeguard for autonomous deployment.

7 Cross-Cutting Defenses and Mitigation Strategies

The defense of large language models (LLMs) has evolved beyond isolated countermeasures for individual threat categories toward cross-cutting strategies that compose alignment hardening, pre- and post-generation filtering, input/output sanitization, adversarial training, and proactive red teaming into resilient multi-layered systems. A defense-in-depth philosophy motivates the introduction of additional independent defense layers that evaluate the unsafety level of produced outputs and perform post-generation filtering, which can serve as an ultimate defense against alignment failures [4]. Complementarily, alignment methods based on Reinforcement Learning from Human Feedback (RLHF) and Direct Preference Optimization (DPO), together with system-level guardrails that monitor prompts and generated text, form foundational layers of a comprehensive defense stack.

7.0.1 Defense-in-Depth and Multi-Layered Strategies

Defense-in-depth leverages multiple independent safeguards so that the failure of one layer can be compensated by another. On the input side, perplexity-based filtering examines prompts for anomalous statistics and discards those that appear adversarial. At the data level, spectral signatures detect poisoned training samples through anomalous internal representations [80]; however, their effectiveness is sensitive to configuration choices, and widely used settings are often suboptimal [36]. Together, these layers reduce the overall attack surface, but even stacked

static defenses can be circumvented by adaptive adversaries, a theme revisited later.

7.0.2 Alignment Hardening and Safety Training

Alignment-based hardening constitutes the inner loop of LLM defense. Supervised fine-tuning on curated safety data provides a useful but incomplete alignment layer. RLHF and DPO further optimize responses from preference data. Liu et al. (2025) applied DPO to safety-oriented preference data from Anthropic-HH and other safety corpora, training models to prefer harmless over harmful responses [45]. Beyond reward-based alignment, instruction hierarchy trains models to prioritize trusted instructions over lower-privilege content [85]. This training-time hardening embeds a persistent priority structure that cannot be easily overridden by injected lower-priority prompts.

7.0.3 Guardrails, Filtering, and Input/Output Sanitization

System-level guardrails act as intermediaries that inspect both incoming prompts and model outputs, preventing obviously malicious content from reaching the model or the end user. For indirect prompt injection, sanitization methods like PISanitizer locate and remove injected tokens by analyzing attention weights across consecutive tokens; they reduce the attack success rate to nearly zero while preserving target task utility [23]. System prompt extraction is countered by appending safety instructions (instruction defense), wrapping the system prompt with double-layered safety instructions (sandwich defense), and applying output filtering that blocks responses containing the original system prompt [17]. At the output level, post-generation filtering that uses an external Judge LLM to intercept and block harmful outputs provides an additional defense layer beyond intrinsic alignment [4]. Nevertheless, a wide array of jailbreaking techniques can bypass these filters, underscoring the need for more fundamental hardening strategies.

7.0.4 Constrained Decoding for Hallucination Prevention

Decoding-time interventions offer a complementary line of defense against factual inaccuracies without retraining. Yang et al. (2025) introduced InEx, a cross-modal verification framework that calibrates logits and applies uncertainty penalties during generation to reduce hallucinations [98]. The absence of certified constrained decoding for hallucination prevention in the retrieved literature highlights an open frontier where robustness against fabricated content must be mathematically grounded.

7.0.5 Adversarial Training and Training-Time Hardening

Adversarial training in the LLM domain often takes the form of fine-tuning on explicitly crafted attack prompts to teach secure behavior. Instruction-hierarchy

training is a representative example: by exposing the model to conflicting lower-privilege content while enforcing a strict priority policy, the resulting model learns to disregard injected instructions [85]. More generally, prevention-based defenses can be implemented at training time, as shown by Jatmo, which fine-tunes a base LLM on a single task without supplying task instructions, thereby removing the instruction-following behavior exploited by prompt injection [65]. These training-time approaches embed security directly into model weights, making them harder to bypass than prompt-level measures.

7.0.6 Red Teaming as a Defensive Feedback Loop

Red teaming has evolved from a manual auditing practice into an integral component of the defense lifecycle. Human red teaming uncovers nuanced weaknesses, while automated red teaming uses adversarial attacks and benchmarks to probe defenses at scale.

7.0.7 Prompt-Level Hardening versus Architectural Defenses

A fundamental distinction in the defense landscape lies between surface-level prompt fortification and deeper weight-level interventions. Prompt-based defenses construct special instructions—such as in-context exemplars or explicit delimiters—without modifying model weights. These methods are lightweight and rapidly deployable, but they operate purely at the textual surface and can be overridden by adversarial prompts that subvert the protective instructions. In contrast, architectural and training-time defenses embed security more deeply. Instruction hierarchy trains the model to enforce a strict priority ordering so that lower-priority injected instructions are systematically disregarded [85]. This complementary relationship—prompt hardening as a rapidly configurable outer shell and architectural fine-tuning as a persistent inner foundation—exemplifies the defense-in-depth philosophy.

7.0.8 Safety–Utility Trade-offs

The tension between safety enforcement and the model's ability to fulfill legitimate requests remains a central challenge. Some defenses explicitly aim to preserve utility: PISanitizer, for instance, reduces attack success rate to nearly zero while maintaining target task performance [23]. However, over-refusal—where benign inputs are incorrectly blocked or refused—is a widely recognized side effect of aggressive alignment and filtering; the retrieved literature does not include dedicated studies that quantify over-refusal rates with the same rigor applied to attack success. Achieving a robust safety–utility balance requires careful calibration of protective mechanisms, and the ability to tune safety parameters without expensive retraining remains a valuable design goal.

7.0.9 Limitations Against Adaptive Multi-Vector Attacks

Despite the layered defenses described above, current state-of-the-art methods remain brittle when faced with adaptive adversaries that combine multiple attack vectors. LLMs remain susceptible to jailbreaking techniques that circumvent safety filters. The absence of structural constraints on agent behavior allows attackers to trigger malicious tool invocations even when the model is guarded by advanced prompting or detection models [57]. These observations indicate that current heuristics are insufficient against persistent, multi-turn, multi-vector attack strategies and motivate the need for provably robust frameworks.

7.0.10 Certified Robustness and Emerging Directions

In the poisoning domain, formal guarantees remain scarce. Lorenz et al. (2024) introduced FullCert, a framework that provides deterministic end-to-end certificates against both training-time poisoning and inference-time evasion attacks by unrolling the entire training and inference procedure into a single computational graph and applying sound interval bounds [51]. They noted, however, that certifying the robustness of training is far more challenging than inference due to the attacker's higher degree of freedom over multiple data points and the much deeper computational graph that spans forward and backward passes over an entire dataset; thus, existing approaches provide only probabilistic guarantees while severely limiting attacker influence to a few samples or pixels [51]. Beyond FullCert, no deterministic certified defenses against arbitrary poisoning have been reported. For jailbreaks and hallucination prevention via constrained decoding, the retrieved literature does not contain certified frameworks, signaling open research frontiers where mathematical verification of safety properties is urgently needed. As attackers continue to develop adaptive, multi-vector strategies, the journey toward robust and generalizable LLM defenses will require not only richer combinations of existing methods but also mathematically grounded frameworks that can provide provable safety guarantees.

Two additional directions reinforce the cross-cutting nature of defense. Sheshadri et al. (2024) observed that adversarial training can fail to fix specific vulnerabilities that were not adversarially trained on, and demonstrated that targeted latent adversarial training—which perturbs latent activations rather than input tokens—improved robustness to unseen jailbreak and backdoor attacks, outperformed a strong baseline with orders of magnitude less compute, and enabled unlearning that resists re-learning [70]. Complementing this, Kumar et al. (2024) introduced AmpleGCG-Plus as an efficient red-teaming tool, showing that model initialization, data quantity, and data quality improve the learning of gibberish adversarial prompts and exposing persistent vulnerabilities in current safety mechanisms [34]. On the guardrail side, Liu et al. (2025) introduced Aegis, which co-evolves attacks and defenses to adapt beyond static, manually designed templates [46]; Yao et al. (2025) similarly motivated adaptive AI firewall designs for multirole system flows

[99]. The retrieved corpus does not document low-rank adaptation regularization as a defense; the closest training-time mechanism is latent adversarial training on hidden activations.

8 Testing Methodologies and Evaluation

Systematic evaluation of large language model (LLM) security relies on carefully curated benchmarks, standardised quantitative metrics, and reproducible evaluation pipelines. This section reviews the empirical apparatus employed across four principal threat categories: prompt injection and agent manipulation, jailbreak attacks, malicious code generation, and hallucination detection with faithfulness verification. Table 1 provides a consolidated overview of the major testing frameworks, metrics, and representative studies. While the survey targets a comprehensive scope, several notable benchmarks – including CWE-BENCH-PYTHON for code security, FaithEval for multi-dimensional faithfulness, and AdvBench for adversarial suffix attacks – are not covered by the retrieved corpus; accordingly, the discussion below is circumscribed to the categories for which direct evaluation evidence is present in the available material.

Table 1: Summary of key testing methodologies across LLM security evaluation categories.

8.0.1 Prompt Injection and Agent Security Evaluation

Prompt injection attacks, especially indirect variants that exploit retrieved or tool-returned content, constitute a rapidly maturing threat. Several benchmarks have been proposed to evaluate the security of LLM-powered agents and retrieval-augmented systems. Widely used suites include AgentDojo, Agent Security Bench, and InjecAgent, which simulate tool-calling and RAG scenarios. Bhagwatkar et al. (2025) identified critical limitations in these benchmarks: many do not model real-world situations appropriately and sometimes use skewed metrics that make weak defenses appear deceptively effective [6]. To remedy this, the authors proposed targeted fixes and constructed a much stronger three-stage cascaded attack strategy that escalates from existing formatted injections to second-order semantic deception templates and finally to a defense-aware adaptive LLM mutator [6]. The authors also released corrected benchmark versions and evaluated a Sanitizer defense, which—not the attack protocol—achieved the reported security-utility trade-off across AgentDojo, Agent Security Bench, InjecAgent, and τ-bench [6].

In parallel, Lee et al. (2026) introduced MPIB, a dataset-and-benchmark suite for evaluating clinical safety under prompt injection in LLM and RAG systems [37]. MPIB covers both direct user-channel injection (V1) and RAG-mediated indirect injection (V2) across clinically grounded scenarios, and reports outcomes using a harm taxonomy together with the Attack Success Rate (ASR) and a novel Clinical Harm Event Rate (CHER) [37]. The benchmark contains 9,697 curated instances

Category	Benchmark / Dataset	Primary Metrics	Tools / Frameworks	Representative Papers
Prompt Injection & Agent Security	AgentDojo, Agent Security Bench, InjecAgent, τ-bench; MPIB (clinical safety)	Attack Success Rate (ASR), Clinical Harm Event Rate (CHER), security-utility trade-off	Cascaded three-stage attack; MPIB evaluation harness	
Jailbreak Attacks	Harmbench (LLM-based ASR classifier); StrongREJECT auto-grading	Attack Success Rate (ASR), StrongREJECT Score, Total Strategies Found (TSF), Average Jailbreak Rounds (AJR)	AutoDAN-Turbo evaluation protocol	[48]
Malicious Code Generation	RMCBench – 473 prompts, text-to-code and code-to-code, 11 malware types, 9 languages	Refusal Rate, Attack Success Rate (ASR), Malicious Ratio (MR)	GPT-4 automatic judge; Code-Jailbreaker attack pipeline	
Hallucination Detection	HaluBench – 15k samples, finance & medicine domains; HaluEval (as sub-component); multimodal benchmarks (Hallusion-Bench, Object-HalBench, MMHal-Bench, AMBER, MMStar)	Accuracy (PASS/FAIL), precision, recall, F1, hallucination rate, question-/figure-/overall accuracy	LYNX hallucination detector; GPT-4o / DeepSeek-V3 evaluators	
Faithfulness & Citation Verification	CogniBench / CogniBench-L (cognitive statements); fine-grained citation-support evaluation (full/partial/no support)	Alignment with human judgments, correlation coefficients, classification accuracy for support levels	Multi-level faithfulness metric framework; automated faithfulness scores	

produced via multi-stage quality gates and clinically informed linting; across a diverse model matrix and five defense configurations, ASR and CHER can diverge, and defense effectiveness depends on whether adversarial instructions appear in the query or in retrieved context [37]. MPIB is accompanied by a reproducible evaluation harness and a redacted release policy for high-risk payloads.

Expanding the scope to multi-agent systems, Triedman et al. (2025) systematically investigated how compromised metadata and "confused deputy" vulnerabilities can hijack control flows in multi-agent architectures, even when individual agents refuse unsafe actions [81]. Their work extends indirect prompt injection (IPI) evaluation beyond single-agent setups, highlighting the need for benchmarks that capture complex agent interactions and tool-augmented workflows.

8.0.2 Jailbreak Evaluation Metrics

Beyond prompt injection, jailbreak attacks that attempt to elicit forbidden outputs from aligned LLMs have been measured through standardised metrics. Liu et al. (2024) employed two evaluation metrics drawn from existing open-source jailbreak benchmarks: the Harmbench Attack Success Rate (ASR), computed by a carefully fine-tuned Llama-2-13B classifier that judges whether a response is both relevant and harmful, and the StrongREJECT Score, an auto-grading system that captures nuanced distinctions in response quality and aligns closely with human evaluators [48]. For iterative attack methods such as AutoDAN-Turbo, the evaluation is supplemented with Total Strategies Found (TSF) and Average Jailbreak Rounds (AJR), providing insight into both attack diversity and efficiency [48]. These metrics have become a reference for jailbreak studies, offering a reproducible quantification of attack effectiveness.

8.0.3 Malicious Code Generation Benchmarks

A critical threat to LLM security is the generation of malicious code. The first benchmark specifically designed to assess this capability is RMCBench, introduced by Chen et al. in 2024 [10]. RMCBench comprises 473 prompts split into two scenarios: a *text-to-code* scenario where natural-language descriptions request code generation, and a *code-to-code* scenario requiring translation or completion of existing malicious code [10]. The dataset covers 11 categories of malware (e.g., viruses, worms) written in 9 programming languages such as Python, Java, and C++ [10]. RMCBench was built from malicious code collected from 392 GitHub repositories; ChatGPT-3.5 generated natural-language descriptions, which the authors used to manually construct the benchmark prompts [10].

The standard evaluation protocol on RMCBench employs GPT-4 as an automatic judge to label each model response as GOOD (refusal to generate), BAD (malicious output), or UNCLEAR, and reports the **Refusal Rate** – the percentage of prompts eliciting a GOOD response [10]. Chen et al.'s original study on 11 representative LLMs revealed an average refusal rate of only 28.71

Building on this, Ouyang et al. (2025) refined the evaluation metrics by introducing **Attack Success Rate (ASR)** – the proportion of malicious requests that are *not* rejected – and **Malicious Ratio (MR)** – the fraction of generated code that faithfully realises the malicious intent [62]. Using the same RMCBench benchmark, their CodeJailbreaker attack achieved an average ASR of nearly 80

8.0.4 Hallucination Detection Benchmarks

Hallucination – the generation of unfaithful or unsupported information – is assessed through a growing array of evaluation suites. Early works included SelfCheckGPT, a sampling-based black-box detector [54], and HaluEval, a benchmark of generated and human-annotated hallucination samples [39]. While useful, these datasets were largely synthetic and lacked domain diversity [68]. In 2024, Ravi et al. introduced **HaluBench**, a benchmark of 15,000 samples that, for the first time, incorporated real-world domains – specifically finance and medicine – making it the first open-source hallucination benchmark spanning specialised industrial tasks [68]. HaluBench combines DROP, HaluEval, RAGTruth, FinanceBench, PubMedQA, and COVIDQA; each instance contains context, a question, an answer, and a PASS/FAIL faithfulness label [68]. The benchmark deliberately includes hard-to-detect hallucinations – answers that appear plausible but contradict the provided context [68]. On this dataset, the LYNX model, a fine-tuned Llama-3-70B-Instruct variant, achieved reference-free hallucination detection superior to GPT-4o, Claude-3-Sonnet, and other closed-source alternatives [68]. Mala et al. (2025) later used HaluBench – with 13,867 samples – to study hybrid mitigation strategies, confirming its utility for comparing detection approaches [52].

Beyond single-turn binary classification, Anaokar et al. (2025) proposed HalluDetect, a structured multi-stage LLM-based evaluation framework designed to detect, score, and explain hallucinations in multi-turn chatbot conversations [2]. HalluDetect was evaluated on the DetectorEval dataset using human annotation; annotators labelled detected hallucinations as Correct or Wrong and reported missed instances, yielding per-conversation precision and recall metrics that offer a finer-grained assessment than simple accuracy [2].

The scope of hallucination evaluation has further broadened to multimodal domains. Liu et al. (2025) evaluated hallucination mitigation on five vision-language benchmarks – HallusionBench, Object-HalBench, MMHal-Bench, AMBER, and MMStar – using question-level accuracy, figure-level accuracy, overall accuracy, and hallucination rates [47].

8.0.5 Faithfulness and Citation Evaluation

Ensuring that generated statements are supported by retrieved documents is critical for retrieval-augmented generation (RAG). Beyond binary hallucination detection, recent work has targeted multi-level faithfulness and citation verification. Tang et al. (2025) proposed **CogniBench**, a benchmark specifically designed to evaluate

faithfulness hallucinations involving "cognitive statements" – claims that require inference from the context rather than mere factual restatement [77]. They further introduced an automatic annotation pipeline to scale the dataset (CogniBench-L) and train detectors capable of handling both factual and cognitive hallucinations [77].

Parallel efforts have focused on the evaluation of citation support. Zhang et al. (2024a, 2024b) examined the use of faithfulness metrics to automatically judge whether a citation fully, partially, or not at all supports a generated statement [105, 106]. Their comparative framework assessed metric performance through correlation with human judgments, classification evaluation, and retrieval evaluation, revealing that no single metric consistently excels across all three support levels [105]. This finding highlights the need for more nuanced, multi-dimensional evaluation schemes that move beyond simple binary pass/fail decisions.

8.0.6 Red-Teaming Frameworks and Automated Evaluation Suites

The surveyed literature illustrates a transition from ad-hoc testing to systematic, automated red-teaming. The cascaded attack strategy of Bhagwatkar et al. (2025) serves as a reusable adversarial probe that can be deployed across multiple agent benchmarks, functioning as a standardised strength evaluator [6]. MPIB provides a fully packaged evaluation harness with fixed splits, a clinical-harm taxonomy, and dual metrics (ASR + CHER), enabling reproducible comparisons across models and defense configurations [37]. For code generation, the CodeJailbreaker pipeline integrates with RMCBench to deliver a structured attack and evaluation workflow [62]. In the jailbreak domain, AutoDAN-Turbo's reporting of TSF and AJR alongside Harmbench ASR and StrongREJECT forms a comprehensive evaluation suite [48]. Together, these components move the community toward reproducible, multi-faceted red-teaming, although a unified, cross-category framework remains absent.

8.0.7 Temporal Evolution and Cross-Category Observations

The benchmarks and metrics described above reveal a clear evolutionary trajectory. Early evaluation concentrated on single-turn safety violations: Harmbench ASR and StrongREJECT (2024) quantified basic jailbreak success, while HaluEval provided synthetic hallucination samples. By 2024–2025, community effort shifted toward realistic, domain-rich settings – HaluBench encompassed healthcare and finance [68], and RMCBench formalised malicious code generation. The period 2025–2026 saw the emergence of multi-turn, tool-augmented, and multi-agent evaluation: AgentDojo, Agent Security Bench, and InjecAgent incorporated tool-calling agents, MPIB introduced clinically grounded indirect injection, and Triedman et al. explored compromised metadata in multi-agent systems [6, 81]. Adaptive attack resistance was explicitly addressed by the cascaded approach of Bhagwatkar et al., which demonstrated that strong attacks can surface vulnerabilities missed by static benchmarks [6]. Multimodal evaluation was added in 2025

with benchmarks like HallusionBench and MMHal-Bench. Hallucination evaluation similarly matured from binary single-turn labelling to multi-turn precision-recall assessment with HalluDetect [2].

A cross-category comparison reveals that metrics are tightly coupled to threat semantics. Prompt injection and jailbreak attacks use ASR to measure the model's ability to gate harmful outputs, with extensions such as CHER for clinical harm and StrongREJECT for response quality. Malicious code generation adds the Malicious Ratio to assess not only whether the model capitulates but also the fidelity of the generated payload. Hallucination detection relies on classifier-style measures – accuracy, precision, recall – that compare outputs against ground-truth faithfulness. This heterogeneity makes direct comparison of security postures across threat vectors difficult and underscores the lack of a unified benchmark that spans all categories under a common evaluation protocol.

8.0.8 Limitations of Current Testing Approaches

Several limitations persist in the current testing methodologies. **Dataset biases** are evident: RMCBench is derived from GitHub repositories and a fixed set of malware categories, potentially missing novel attack patterns [10]. HaluBench, though diverse, is drawn from specific English-centric QA collections and does not capture the full linguistic variety of deployment settings. **Static benchmarks** fail to capture adaptive attack resistance. Bhagwatkar et al. (2025) showed that introducing a defense-aware adaptive attacker dramatically reduces the apparent safety of tested agents, exposing weaknesses that original static tests miss. Similarly, the block-level completion results of Ouyang et al. (2025), where even a baseline attacker achieved over 96

9 Discussion and Open Challenges

The rapid advancement of large language models has been paralleled by an equally dynamic threat landscape, yet the security community remains fragmented along isolated attack vectors rather than holistic resilience. This section synthesizes insights across prompt injection, data poisoning, and adversarial attacks to identify critical gaps and propose directions for future research.

9.0.1 Certified Defenses and the Poisoning Impasse

Lorenz et al. observe that the first works on certified defenses with probabilistic guarantees underscore the profound difficulty of the problem, and multiple structural factors—such as the attacker's ability to influence many training points and the deep computational graph of training—make training-time certification far harder than inference-time certification [51]. In practice, poisoning attacks on LLM-based systems can exploit external knowledge sources; Wang et al. demonstrated that LLM agents can be backdoored via triggers injected into training data,

enabling malicious operations when the trigger appears in the input or environment [88]. Li et al.'s AutoBackdoor framework further automates backdoor injection for LLMs, achieving over 90

9.0.2 The Fragmentation of Benchmarks

Security evaluations today are largely compartmentalized: individual benchmarks assess jailbreaking, prompt injection, or backdoor resilience in isolation, failing to capture the multi-vector strategies that real-world adversaries employ. Tejedor et al. explicitly argue that ensuring robustness requires standardized evaluation benchmarks and threat models that cover jailbreaks, prompt injections, and data poisoning [78]. Meanwhile, agent-centric benchmarks including InjecAgent, AgentDojo, and Agent Security Bench evaluate indirect prompt injection, but they frequently lack realistic environments and remain partly private, which hinders reproducibility and the emergence of a unified risk perspective [103]. Indirect prompt injection in tool-integrated agents has been systematically benchmarked in suites such as InjecAgent and AgentDojo, confirming the severity of such risks [30]. A comprehensive benchmark that concurrently measures robustness against jailbreaking, prompt injection, data poisoning, and hallucination would enable researchers to detect cross-dimensional trade-offs, such as whether a defense that hardens against indirect injection inadvertently increases susceptibility to backdoor triggers. Realizing such a benchmark requires not only technical integration but also community-wide agreement on threat models and evaluation protocols, a standardization effort that has yet to materialize.

9.0.3 Multimodal Attack Surfaces

The expansion of LLMs to encompass visual, auditory, and embodied modalities dramatically enlarges the attack surface. Tejedor et al. caution that each additional modality introduces unique vulnerabilities that interact with textual threats [78]. Black et al. anticipate that multi-modal models and agentic systems will introduce new categories of misuse beyond those studied to date, raising concerns about the interconnectedness of tools where outputs from one model become inputs to another (the "daisy-chain" problem) [7]. They highlight that risk assessment of a single component is often insufficient to capture risks that emerge across the wider ecosystem [7]. In the embodied domain, BadVLA demonstrates that vision-language-action models are vulnerable to untargeted backdoors that cause widespread behavioral deviation [110]. These findings underscore that developing safeguards at the fusion points of multimodal architectures is an urgent open challenge, and that certification must jointly consider all input modalities.

9.0.4 Securing Agentic Frameworks Against Cascading Failures

The shift from isolated chatbots to tool-using, multi-agent systems introduces qualitatively new failure modes where individually benign actions can combine into

harmful outcomes. Li et al. provided a taxonomy of agent-specific attacks and demonstrated that exploiting memory, retrieval, and API components requires no sophisticated machine learning expertise [38]. Liu et al. showed that agentic AI coding editors can be manipulated to execute dangerous actions without user awareness, expanding the damage potential of prompt injection [50]. The most acute threat comes from multi-turn, indirect injection: Li et al.'s Sequential Tool Attack Chaining (STAC) demonstrated that individually harmless tool calls can be chained to perform collectively harmful actions, with attack success rates exceeding 90

9.0.5 Transparency, White-Box Access, and Ethical Disclosure

A persistent tension exists between the benefits of model transparency for auditing and the adversarial advantage conferred by white-box knowledge. The ethical implications of releasing attack tools are particularly acute. Das et al. emphasize that exposing system prompt extraction attacks and proposing defenses raises awareness and catalyzes further research on trustworthy LLMs [17]. Zhan et al. argue that revealing vulnerabilities through rigorous research is essential for building defenses and aligns with ethical principles, especially when the techniques are simple enough to be independently discovered by adversaries [102]. Weng et al. emphasize that transparent research is necessary to develop better protections and that responsible disclosure enables AI developers to proactively address vulnerabilities [93]. Several works have explicitly framed their tools as defensive red-teaming instruments: AutoBackdoor is intended to identify and mitigate backdoor vulnerabilities [43], REDCODER is designed strictly for uncovering flaws in code generation models [58], and RAINBOWPLUS is released with guidelines encouraging ethical use and collaboration with developers [16]. Nonetheless, the community still lacks universally accepted guidelines for responsible disclosure of attack methodologies and exploitation code. Establishing norms analogous to responsible vulnerability disclosure in cybersecurity could mitigate dual-use risks while enabling defensive progress, but formalizing such norms remains an unresolved governance gap.

9.0.6 Regulatory Standards and the Missing Temporal Dimension

A conspicuous absence in the current LLM security literature is the development or consideration of regulatory standards for deployment assurance. Existing works overwhelmingly concentrate on technical attack–defense dynamics without engaging with the formulation or enforcement of security requirements, leaving the field vulnerable to reactive, post-incident regulation. Moreover, the dominant focus on point-in-time evaluations obscures the evolution of attacks over time: longitudinal studies that track how adversary strategies mutate in response to model updates, data shifts, or patching are virtually nonexistent. The aspirational goal of self-healing models—LLM-based systems that can autonomously detect, contain, and recover from adversarial manipulations—remains entirely unexplored. Addressing these deficits will require interdisciplinary collaboration spanning machine learn-

ing, systems security, and public policy, moving the community from reactive hardening toward proactive, certifiable, and continuously adaptive defense paradigms.

Taken together, the findings in this survey reveal a gradual convergence of attack taxonomies, yet defensive research remains siloed. One notable step toward integration is UniGuardian, which reframes prompt injection, backdoor attacks, and adversarial attacks as prompt-trigger attacks and introduces a single-forward detection pipeline capable of flagging all three in one pass [44]. Extending such unified detection to multimodal and agentic deployments is an open challenge. Benchmark development faces similar fragmentation: Bhagwatkar et al. identified key flaws in popular agent-security benchmarks such as AgentDojo, Agent Security Bench, and InjecAgent, and released corrected versions alongside baselines [6]. This suggests that future evaluations should move toward adaptive, defense-aware attack protocols and ensure that novel frameworks are validated against corrected benchmarks, rather than inherited suites, before being adopted by practitioners.

10 Conclusion

10.0.1 Synthesis of Threat Evolution

Sandoval et al. (2025) reported a 31

10.0.2 The Defensive Arms Race

Early adversarial fine-tuning treatments were shown to suppress rather than remove harmful capabilities, leaving models vulnerable to unseen jailbreaks [70]. Today, defense approaches are broadly categorized into detection-based and prevention-based methods. Co-evolutionary frameworks such as Aegis systematically evolve attackers and defenders together, achieving adaptive robustness against prompt injection [46]. PromptSleuth introduced a detection mechanism that maintains robustness across diverse and evasive attack variants, significantly lowering undetected attack rates while adding only modest overhead [87]. To address residual alignment weaknesses, SecureBreak provides a manually annotated dataset for building post-generation filtering modules that act as a final guardrail, and can also guide alignment optimization [4]. For jailbreaks, adaptive activation steering dynamically adjusts refusal strength based on input characteristics, overcoming the static limitations of prior approaches [108]. At the frontier of robustness certification, Lorenz et al. (2024) introduced FullCert, a deterministic certifier against both data-poisoning and evasion attacks, accompanied by formal convergence analysis and an open-source toolset [51]. VeriGuard complements such certification by formally verifying agent policies against safety specifications, achieving near-zero attack success rates in benchmark evaluations [57]. Adaptive attacks have circumvented every prompt-injection defense evaluated on LLM agents, confirming that no single technique is impregnable [102].

10.0.3 The Critical Role of Benchmarks

Wahed et al. (2025) introduced MOCHA, a benchmark for assessing code LLMs against multi-turn malicious prompts, revealing high vulnerability to decomposed harmful intent [84]. Das et al. (2025) introduced SPE-LLM, a framework for systematically assessing system prompt extraction attacks, demonstrating that simple safety instructions are insufficient to prevent intellectual property leakage [17].

10.0.4 Practical Recommendations

For practitioners, three actionable principles emerge. First, implement defense-in-depth: no single mechanism suffices, so robust deployments should combine prevention, real-time detection, and post-generation filtering. Second, evaluate against adaptive, multi-turn attacks rather than static benchmarks alone. Third, demand verifiable safety claims through standardized scores, explicit threat models, and, where feasible, certification reports before integrating LLMs into safety-critical applications.

10.0.5 Toward a Secure LLM Ecosystem

The evolution of LLM safety research points toward certified robustness, exemplified by deterministic end-to-end frameworks such as FullCert [51]. Extending deterministic certification to the scale and multimodality of modern LLMs, integrating automated red-teaming agents into continuous testing loops, and updating guardrails in real time will bridge the gap between static evaluation and dynamic threats. A secure LLM ecosystem will be built on comprehensive, verifiable safety engineering that keeps pace with rapid model advancement.

While much of the literature focuses on model-level attacks, the evaluation infrastructure itself has become an attack surface. Leaderboard maintainers have adopted complementary measures, including monitoring query distributions and community flagging mechanisms, to raise the cost of adversarial manipulation [76]. Likewise, performance benchmarks establish realistic expectations for security-performance trade-offs in serverless ML deployments, enabling organizations to calibrate security controls to their risk profiles and performance requirements [63]. Building on these practical lessons, alignment methods such as Safe RLHF still fall short for high-stakes contexts because they lack formal guarantees, and Seldonian probabilistic guarantees on constraint satisfaction offer a principled path toward stronger assurances [14]. Together, these observations reinforce the central thesis that a secure LLM ecosystem depends not only on robust models but also on trustworthy evaluation and verifiable deployment practices.

---

11 References

See the accompanying `references.bib` *file for the complete bibliography.*

References

[1] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji. Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 1023–1039. Association for Computational Linguistics, 2025.

[2] S. Anaokar, S. Ganatra, H. Kashid, S. Bhattacharyya, S. Nair, R. Sekhar, S. Manohar, R. Hemrajani, and P. Bhattacharyya. Halludetect: Detecting, mitigating, and benchmarking hallucinations in conversational systems in the legal domain. In S. Potdar, L. M. Rojas-Barahona, and S. Montella, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025 - Industry Track, Suzhou, China, November 4-9, 2025*, pages 1822–1847. Association for Computational Linguistics, 2025.

[3] C. Anil, E. Durmus, N. Panickssery, M. Sharma, J. Benton, et al. Many-shot Jailbreaking. In *Advances in Neural Information Processing Systems 37*, 2024.

[4] M. Arazzi, V. K. Kembu, and A. Nocera. Securebreak - A dataset towards safe and secure models. *CoRR*, abs/2603.21975, 2026.

[5] T. Baumgärtner, Y. Gao, D. Alon, and D. Metzler. Best-of-venom: Attacking RLHF by injecting poisoned preference data. *CoRR*, abs/2404.05530, 2024.

[6] R. Bhagwatkar, K. Kasa, A. Puri, G. Huang, I. Rish, G. W. Taylor, K. D. Dvijotham, and A. Lacoste. Indirect prompt injections: Are firewalls all you need, or stronger benchmarks? *CoRR*, abs/2510.05244, 2025.

[7] J. R. M. Black, M. S. Hanke, A. Maiwald, T. Hernandez-Boussard, O. M. Crook, and J. Pannu. Open-weight genome language model safeguards: Assessing robustness via adversarial fine-tuning. *CoRR*, abs/2511.19299, 2025.

[8] H. Chang, Y. Jun, and H. Lee. Chatinject: Abusing chat templates for prompt injection in LLM agents. *CoRR*, abs/2509.22830, 2025.

[9] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong. Jailbreaking Black Box Large Language Models in Twenty Queries. *CoRR*, abs/2310.08419, 2023.

[10] J. Chen, Q. Zhong, Y. Wang, K. Ning, Y. Liu, Z. Xu, Z. Zhao, T. Chen, and Z. Zheng. Rmcbench: Benchmarking large language models' resistance to malicious code. In V. Filkov, B. Ray, and M. Zhou, editors, *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*, pages 995–1006. ACM, 2024.

[11] S. Chen, J. Piet, C. Sitawarin, and D. Wagner. StruQ: Defending Against Prompt Injection with Structured Queries. *CoRR*, abs/2402.06363, 2024.

[12] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo. SecAlign: Defending Against Prompt Injection with Preference Optimization. *CoRR*, abs/2410.05451, 2024.

[13] Y. Chen, H. Li, Y. Sui, Y. He, Y. Liu, Y. Song, and B. Hooi. Can indirect prompt injection attacks be detected and removed? In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 18189–18206. Association for Computational Linguistics, 2025.

[14] Y. Chittepu, B. Metevier, W. Schwarzer, A. Hoag, S. Niekum, and P. S. Thomas. Reinforcement learning from human feedback with high-confidence safety constraints. *CoRR*, abs/2506.08266, 2025.

[15] Y. Cui, S. Pan, Y. Liu, H. Zhang, and C. Zuo. Vortexpia: Indirect prompt injection attack against llms for efficient extraction of user privacy. *CoRR*, abs/2510.04261, 2025.

[16] Q. Dang, C. Ngo, and T. Hy. Rainbowplus: Enhancing adversarial prompt generation via evolutionary quality-diversity search. *CoRR*, abs/2504.15047, 2025.

[17] B. C. Das, M. H. Amini, and Y. Wu. System prompt extraction attacks and defenses in large language models. *CoRR*, abs/2505.23817, 2025.

[18] K. Duan, H. Yao, Y. Chen, Z. Li, T. Qiao, Z. Qin, and C. Wang. Badreward: Clean-label poisoning of reward models in text-to-image RLHF. *CoRR*, abs/2506.03234, 2025.

[19] S. K. Dutta, Y. Xu, P. Pant, and X. Zhang. GREAT: generalizable backdoor attacks in RLHF via emotion-aware trigger synthesis. *CoRR*, abs/2510.09260, 2025.

[20] S. Egler, J. Schulman, and N. Carlini. Detecting adversarial fine-tuning with auditing agents. *CoRR*, abs/2510.16255, 2025.

[21] P. Elchafei and M. Abu-Elkheir. Hallucination detectives at semeval-2025 task 3: Span-level hallucination detection for llm-generated answers. In S. Rosenthal, A. Rosá, D. Ghosh, and M. Zampieri, editors, *Proceedings of the 19th International Workshop on Semantic Evaluation, SemEval@ACL 2025, Vienna, Austria, July 31 - August 1, 2025*, pages 601–606. Association for Computational Linguistics, 2025.

[22] H. Foerster, I. Shumailov, Y. Zhao, H. Chaudhari, J. Hayes, R. Mullins, and Y. Gal. Reasoning introduces new poisoning attacks yet makes them more complicated. *CoRR*, abs/2509.05739, 2025.

[23] R. Geng, Y. Wang, C. Yin, M. Cheng, Y. Chen, and J. Jia. Pisanitizer: Preventing prompt injection to long-context llms via prompt sanitization. *CoRR*, abs/2511.10720, 2025.

[24] Y. Gu, Z. Ji, W. Zhang, C. Lyu, D. Lin, and K. Chen. Anah-v2: Scaling analytical hallucination annotation of large language models. In A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[25] B. Hou, Y. Zhang, J. Andreas, and S. Chang. A probabilistic framework for LLM hallucination detection via belief tree propagation. *CoRR*, abs/2406.06950, 2024.

[26] Y. Huang, S. Gupta, M. Xia, K. Li, and D. Chen. Catastrophic Jailbreak of Open-source LLMs via Exploiting Generation. *CoRR*, abs/2310.06987, 2023.

[27] E. Hubinger, C. Denison, J. Mu, M. Lambert, M. Tong, et al. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. *CoRR*, abs/2401.05566, 2024.

[28] C. Improta. Poisoning programs by un-repairing code: Security concerns of ai-generated code. In *34th IEEE International Symposium on Software Reliability Engineering, ISSRE 2023 - Workshops, Florence, Italy, October 9-12, 2023*, pages 128–131. IEEE, 2023.

[29] D. Ivry and O. Nahum. Sentinel: SOTA model to protect against prompt injections. *CoRR*, abs/2506.05446, 2025.

[30] F. Jia, T. Wu, X. Qin, and A. C. Squicciarini. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August*

1, 2025, pages 29680–29697. Association for Computational Linguistics, 2025.

[31] H. Kim, H. Na, and D. Choi. Beyond fixed and dynamic prompts: Embedded jailbreak templates for advancing LLM security. *CoRR*, abs/2511.14140, 2025.

[32] S. Kim and G. Lee. Merging triggers, breaking backdoors: Defensive poisoning for instruction-tuned language models. In M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2026, San Diego, California, United States, July 2-7, 2026*, pages 24269–24287. Association for Computational Linguistics, 2026.

[33] T. Krauß, H. Dashtbani, and A. Dmitrienko. Twinbreak: Jailbreaking LLM security alignments based on twin prompts. In L. Bauer and G. Pellegrino, editors, *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, pages 2343–2362. USENIX Association, 2025.

[34] V. Kumar, Z. Liao, J. Jones, and H. Sun. Amplegcg-plus: A strong generative model of adversarial suffixes to jailbreak llms with higher success rates in fewer attempts. *CoRR*, abs/2410.22143, 2024.

[35] S. Kusaka, K. Saito, M. Kudo, T. Tanabe, A. Wachi, and Y. Akimoto. Cost-minimized label-flipping poisoning attack to LLM alignment. *CoRR*, abs/2511.09105, 2025.

[36] Q. H. Le, T. Le-Cong, B. Le, and B. Xu. Signature in code backdoor detection, how far are we? *CoRR*, abs/2510.13992, 2025.

[37] J. Lee, H. Jang, and K. S. Choi. MPIB: A benchmark for medical prompt injection attacks and clinical safety in llms. *CoRR*, abs/2602.06268, 2026.

[38] A. Li, Y. Zhou, V. C. Raghuram, T. Goldstein, and M. Goldblum. Commercial LLM agents are already vulnerable to simple yet dangerous attacks. *CoRR*, abs/2502.08586, 2025.

[39] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. *CoRR*, abs/2305.11747, 2023.

[40] J. Li, J. He, C. Shang, D. Kulshreshtha, X. Xian, Y. Zhang, H. Su, S. Swamy, and Y. Qi. STAC: when innocent tools form dangerous chains to jailbreak LLM agents. *CoRR*, abs/2509.25624, 2025.

[41] J. Li, G. Tu, S. Cheng, J. Hu, J. Wang, R. Chen, Z. Zhou, and D. Shan. LLM hallucination detection: A fast fourier transform method based on hidden layer temporal signals. *CoRR*, abs/2509.13154, 2025.

[42] Y. Li, T. Li, K. Chen, J. Zhang, S. Liu, W. Wang, T. Zhang, and Y. Liu. Badedit: Backdooring large language models by model editing. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[43] Y. Li, Z. Li, W. Zhao, N. M. Min, H. Huang, X. Ma, and J. Sun. Autobackdoor: Automating backdoor attacks via LLM agents. *CoRR*, abs/2511.16709, 2025.

[44] H. Lin, Y. Lao, T. Geng, T. Yu, and W. Zhao. Uniguardian: A unified defense for detecting prompt injection, backdoor attacks and adversarial attacks in large language models. *CoRR*, abs/2502.13141, 2025.

[45] S. Liu, Q. Sheng, D. Wang, Y. Liu, G. Yang, and J. Cao. Forewarned is forearmed: Pre-synthesizing jailbreak-like instructions to enhance LLM safety guardrail to potential attacks. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 4957–4974. Association for Computational Linguistics, 2025.

[46] T. Liu, C. Y. Hsu, K. Lee, C. Fu, and H. Lee. AEGIS : Automated co-evolutionary framework for guarding prompt injections schema. *CoRR*, abs/2509.00088, 2025.

[47] W. Liu, X. Song, J. Li, Y. Wei, N. Zheng, J. Yin, and L. Nie. Mitigating hallucination through theory-consistent symmetric multimodal preference optimization. In D. Belgrave, C. Zhang, L. N. Montoya, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, N. Chen, I. V. M. Ruíz, and A. Loaiza-Bonilla, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, 2025.

[48] X. Liu, P. Li, G. E. Suh, Y. Vorobeychik, Z. Mao, S. Jha, P. McDaniel, H. Sun, B. Li, and C. Xiao. Autodan-turbo: A lifelong agent for strategy self-exploration to jailbreak llms. *CoRR*, abs/2410.05295, 2024.

[49] X. Liu, N. Xu, M. Chen, and C. Xiao. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. *CoRR*, abs/2310.04451, 2023.

[50] Y. Liu, Y. Zhao, Y. Lyu, T. Zhang, H. Wang, and D. Lo. "your ai, my shell": Demystifying prompt injection attacks on agentic AI coding editors. *CoRR*, abs/2509.22040, 2025.

[51] T. Lorenz, M. Kwiatkowska, and M. Fritz. Fullcert: Deterministic end-to-end certification for training and inference of neural networks. In D. Cre-

mers, Z. Lähner, M. Moeller, M. Nießner, B. Ommer, and R. Triebel, editors, *Pattern Recognition - 46th DAGM German Conference, DAGM GCPR 2024, Munich, Germany, September 10-13, 2024, Proceedings, Part I*, volume 15297 of *Lecture Notes in Computer Science*, pages 71–85. Springer, 2024.

[52] C. S. Mala, G. Gezici, and F. Giannotti. Hybrid retrieval for hallucination mitigation in large language models: A comparative analysis. *CoRR*, abs/2504.05324, 2025.

[53] M. A. A. Mamun, I. Alouani, and N. B. Abu-Ghazaleh. Poison once, refuse forever: Weaponizing alignment for injecting bias in llms. *CoRR*, abs/2508.20333, 2025.

[54] P. Manakul, A. Liusie, and M. J. F. Gales. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *CoRR*, abs/2303.08896, 2023.

[55] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. *CoRR*, abs/2402.04249, 2024.

[56] A. Mehrotra, M. Zampetakis, P. Kassianik, B. Nelson, H. Anderson, Y. Singer, and A. Karbasi. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically. *CoRR*, abs/2312.02119, 2023.

[57] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, and L. T. Le. Veriguard: Enhancing LLM agent safety via verified code generation. *CoRR*, abs/2510.05156, 2025.

[58] W. J. Mo, Q. Liu, X. Wen, D. Jung, H. Askari, W. Zhou, Z. Zhao, and M. Chen. Redcoder: Automated multi-turn red teaming for code llms. *CoRR*, abs/2507.22063, 2025.

[59] C. Moslonka, H. Randrianarivo, A. Garnier, and E. Malherbe. Learned hallucination detection in black-box llms using token-level entropy production rate. *CoRR*, abs/2509.04492, 2025.

[60] F. Mu, J. Wang, Z. Yu, L. Shi, S. Wang, M. Li, and Q. Wang. Codepurify: Defend backdoor attacks on neural code models via entropy-based purification. *CoRR*, abs/2410.20136, 2024.

[61] A. Nika, J. Nöther, D. Mandal, P. Kamalaruban, A. Singla, and G. Radanovic. Policy teaching via data poisoning in learning from human preferences. In Y. Li, S. Mandt, S. Agrawal, and M. E. Khan, editors, *International Conference on Artificial Intelligence and Statistics, AISTATS 2025,*

Mai Khao, Thailand, 3-5 May 2025, volume 258 of *Proceedings of Machine Learning Research*, pages 1117–1125. PMLR, 2025.

[62] S. Ouyang, Y. Qin, B. Lin, L. Chen, X. Mao, and S. Wang. Smoke and mirrors: Jailbreaking llm-based code generation via implicit malicious prompts. *CoRR*, abs/2503.17953, 2025.

[63] C. Pathade, V. Dhimam, S. Ahmad, and I. Lareb. Serverless AI security: Attack surface analysis and runtime protection mechanisms for faas-based machine learning. *CoRR*, abs/2601.11664, 2026.

[64] A. Paulus, A. Zharmagambetov, C. Guo, B. Amos, and Y. Tian. AdvPrompter: Fast Adaptive Adversarial Prompting for LLMs. *CoRR*, abs/2404.16873, 2024.

[65] J. Piet, M. Alrashed, C. Sitawarin, S. Chen, Z. Wei, E. Sun, B. Alomair, and D. Wagner. Jatmo: Prompt Injection Defense by Task-Specific Finetuning. *CoRR*, abs/2312.17673, 2023.

[66] X. Qi, A. Panda, K. Lyu, X. Ma, S. Roy, A. Beirami, P. Mittal, and P. Henderson. Safety Alignment Should Be Made More Than Just a Few Tokens Deep. *CoRR*, abs/2406.05946, 2024.

[67] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, and P. Henderson. Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To! *CoRR*, abs/2310.03693, 2023.

[68] S. S. Ravi, B. Mielczarek, A. Kannappan, D. Kiela, and R. Qian. Lynx: An open source hallucination evaluation model. *CoRR*, abs/2407.08488, 2024.

[69] A. Shelmanov, E. Fadeeva, A. Tsvigun, I. Tsvigun, Z. Xie, I. Kiselev, N. Daheim, C. Zhang, A. Vazhentsev, M. Sachan, P. Nakov, and T. Baldwin. A head to predict and a head to question: Pre-trained uncertainty quantification heads for hallucination detection in LLM outputs. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 35712–35731. Association for Computational Linguistics, 2025.

[70] A. Sheshadri, A. Ewart, P. Guo, A. Lynch, C. Wu, V. Hebbar, H. Sleight, A. C. Stickland, E. Perez, D. Hadfield-Menell, and S. Casper. Targeted latent adversarial training improves robustness to persistent harmful behaviors in llms. *CoRR*, abs/2407.15549, 2024.

[71] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun. Prompt injection attack to tool selection in LLM agents. *CoRR*, abs/2504.19793, 2025.

[72] T. Shi, K. Zhu, Z. Wang, Y. Jia, W. Cai, W. Liang, H. Wang, H. Alzahrani, J. Lu, K. Kawaguchi, B. Alomair, X. Zhao, W. Y. Wang, N. Gong, W. Guo, and D. Song. Promptarmor: Simple yet effective prompt injection defenses. *CoRR*, abs/2507.15219, 2025.

[73] Y. Shih and Y. Kang. Design and implementation of a secure rag-enhanced AI chatbot for smart tourism customer service: Defending against prompt injection attacks - A case study of hsinchu, taiwan. *CoRR*, abs/2509.21367, 2025.

[74] A. Souly, Q. Lu, D. Bowen, T. Trinh, E. Hsieh, S. Pandey, P. Abbeel, J. Svegliato, S. Emmons, O. Watkins, and S. Toyer. A StrongREJECT for Empty Jailbreaks. *CoRR*, abs/2402.10260, 2024.

[75] A. Souly, J. Rando, E. Chapman, X. Davies, B. Hasircioglu, E. Shereen, C. Mougan, V. Mavroudis, E. Jones, C. Hicks, N. Carlini, Y. Gal, and R. Kirk. Poisoning attacks on llms require a near-constant number of poison samples. *CoRR*, abs/2510.07192, 2025.

[76] A. Suri, H. Chaudhari, Y. Peng, A. Naseh, A. Houmansadr, and A. Oprea. Exploiting leaderboards for large-scale distribution of malicious models. *CoRR*, abs/2507.08983, 2025.

[77] X. Tang, J. Li, K. Hu, N. Du, X. Li, X. Zhang, W. Sun, and S. Xie. Cognibench: A legal-inspired framework and dataset for assessing cognitive faithfulness of large language models. In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 21567–21585. Association for Computational Linguistics, 2025.

[78] V. Tejedor, C. Zuheros, C. Peláez-González, D. Herrera-Poyatos, A. Herrera-Poyatos, and F. Herrera. The h-elena trojan virus to infect model weights: A wake-up call on the security risks of malicious fine-tuning. *CoRR*, abs/2504.03823, 2025.

[79] C. Tong, Q. Zhang, J. Gao, L. Jiang, Y. Liu, and N. Sun. Halunet: Multi-granular uncertainty modeling for efficient hallucination detection in LLM question answering. *CoRR*, abs/2512.24562, 2025.

[80] B. Tran, J. Li, and A. Madry. Spectral Signatures in Backdoor Attacks. *CoRR*, abs/1811.00636, 2018.

[81] H. Triedman, R. D. Jha, and V. Shmatikov. Multi-agent systems execute arbitrary malicious code. *CoRR*, abs/2503.12188, 2025.

[82] S. O. ul Islam, A. Lauscher, and G. Glavas. How much do llms hallucinate across languages? on realistic multilingual estimation of LLM hallucination. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 29077–29098. Association for Computational Linguistics, 2025.

[83] A. Urlana, G. Kanumolu, C. V. Kumar, B. M. Garlapati, and R. Mishra. Hallucounter: Reference-free LLM hallucination detection in the wild! In K. Inui, S. Sakti, H. Wang, D. F. Wong, P. Bhattacharyya, B. Banerjee, A. Ekbal, T. Chakraborty, and D. P. Singh, editors, *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, IJCNLP-AACL 2025, Mumbai, India, December 20-24, 2025*, pages 352–383. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics, 2025.

[84] M. Wahed, X. Zhou, K. A. Nguyen, T. Yu, N. Diwan, G. Wang, D. Hakkani-Tür, and I. Lourentzou. MOCHA: are code language models robust against multi-turn malicious coding prompts? In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 22922–22948. Association for Computational Linguistics, 2025.

[85] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *CoRR*, abs/2404.13208, 2024.

[86] J. Wang, J. Wu, M. Chen, Y. Vorobeychik, and C. Xiao. RLHFPoison: Reward Poisoning Attack for Reinforcement Learning with Human Feedback in Large Language Models. *CoRR*, abs/2311.09641, 2023.

[87] M. Wang, Y. Zhang, and G. Gu. Promptsleuth: Detecting prompt injection via semantic intent invariance. *CoRR*, abs/2508.20890, 2025.

[88] Y. Wang, D. Xue, S. Zhang, and S. Qian. Badagent: Inserting and activating backdoor attacks in LLM agents. In L. Ku, A. Martins, and V. Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 9811–9827. Association for Computational Linguistics, 2024.

[89] Z. Wang, H. Li, R. Zhang, W. Jiang, and G. Xu. Badtemplate: A training-free backdoor attack via chat template against large language models. *CoRR*, abs/2602.05401, 2026.

[90] Z. Wang, J. Liu, S. Zhang, and Y. Yang. Poisoned langchain: Jailbreak llms by langchain. *CoRR*, abs/2406.18122, 2024.

[91] Z. Wang, N. Nagaraja, L. Zhang, H. Bahsi, P. Patil, and P. Liu. To protect the LLM agent against the prompt injection attack with polymorphic prompt. In *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2025 - Supplemental Volume, Naples, Italy, June 23-26, 2025*, pages 22–28. IEEE, 2025.

[92] T. Wen, C. Wang, X. Yang, H. Tang, Y. Xie, L. Lyu, Z. Dou, and F. Wu. Defending against indirect prompt injection by instruction detection. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 19472–19487. Association for Computational Linguistics, 2025.

[93] Z. Weng, X. Jin, J. Jia, and X. Zhang. Foot-in-the-door: A multi-turn jailbreak for llms. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 1939–1950. Association for Computational Linguistics, 2025.

[94] S. Wu, Y. Wang, and Q. Yao. Self-generative adversarial fine-tuning for large language models. *CoRR*, abs/2602.01137, 2026.

[95] T. Wu, S. Zhang, K. Song, S. Xu, S. Zhao, R. Agrawal, S. R. Indurthi, C. Xiang, P. Mittal, and W. Zhou. Instructional segment embedding: Improving LLM safety with instruction hierarchy. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[96] S. Yan, S. Wang, Y. Duan, H. Hong, K. Lee, D. Kim, and Y. Hong. An llm-assisted easy-to-trigger backdoor attack on code completion models: Injecting disguised vulnerabilities against strong detection. In D. Balzarotti and W. Xu, editors, *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*. USENIX Association, 2024.

[97] W. Yang, X. Bi, Y. Lin, S. Chen, J. Zhou, and X. Sun. Watch out for your agents! investigating backdoor threats to llm-based agents. In A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[98] Z. Yang, Y. Yuan, X. Jiang, B. An, and W. Pang. Inex: Hallucination mitigation via introspection and cross-modal multi-agent collaboration. *CoRR*, abs/2512.02981, 2025.

[99] H. Yao, H. Shi, Y. Chen, Y. Jiang, C. Wang, and Z. Qin. Controlnet: A firewall for rag-based LLM system. *CoRR*, abs/2504.09593, 2025.

[100] M. Yin, J. Zhang, J. Sun, M. Fang, H. Li, and Y. Chen. Lobam: Lora-based backdoor attack on model merging. *CoRR*, abs/2411.16746, 2024.

[101] Z. Ying, A. Liu, T. Zhang, Z. Yu, S. Liang, X. Liu, and D. Tao. Jailbreak vision language models via bi-modal adversarial prompt. *CoRR*, abs/2406.04031, 2024.

[102] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In L. Chiruzzo, A. Ritter, and L. Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, volume NAACL 2025 of *Findings of ACL*, pages 7101–7117. Association for Computational Linguistics, 2025.

[103] K. Zhang, M. Tenenholtz, K. Polley, J. Ma, D. Yarats, and N. Li. Browsesafe: Understanding and preventing prompt injection within AI browser agents. *CoRR*, abs/2511.20597, 2025.

[104] S. Zhang, T. Yu, and Y. Feng. TruthX: Alleviating Hallucinations by Editing Large Language Models in Truthful Space. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[105] W. Zhang, M. Aliannejadi, J. Pei, Y. Yuan, J. Huang, and E. Kanoulas. A comparative analysis of faithfulness metrics and humans in citation evaluation. *CoRR*, abs/2408.12398, 2024.

[106] W. Zhang, M. Aliannejadi, Y. Yuan, J. Pei, J. Huang, and E. Kanoulas. Towards fine-grained citation evaluation in generated text: A comparative analysis of faithfulness metrics. In S. Mahamood, M. L. Nguyen, and D. Ippolito, editors, *Proceedings of the 17th International Natural Language Generation Conference, INLG 2024, Tokyo, Japan, September 23 - 27, 2024*, pages 427–439. Association for Computational Linguistics, 2024.

[107] Y. Zhang, J. Rando, I. Evtimov, J. Chi, E. M. Smith, N. Carlini, F. Tramèr, and D. Ippolito. Persistent pre-training poisoning of llms. *CoRR*, abs/2410.13722, 2024.

[108] W. Zhao, J. Guo, Y. Hu, Y. Deng, A. Zhang, X. Sui, X. Han, Y. Zhao, B. Qin, T. Chua, and T. Liu. Adasteer: Your aligned LLM is inherently an adaptive jailbreak defender. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and

V. Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 24559–24577. Association for Computational Linguistics, 2025.

[109] X. Zheng, T. Pang, C. Du, Q. Liu, J. Jiang, and M. Lin. Improved few-shot jailbreaking can circumvent aligned language models and their defenses. In A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[110] X. Zhou, G. Tie, G. Zhang, H. Wang, P. Zhou, and L. Sun. Badvla: Towards backdoor attacks on vision-language-action models via objective-decoupled optimization. In D. Belgrave, C. Zhang, L. N. Montoya, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, N. Chen, I. V. M. Ruíz, and A. Loaiza-Bonilla, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, 2025.

[111] P. Zhu, Z. Zhou, Y. Zhang, S. Yan, K. Wang, and S. Su. Demonagent: Dynamically encrypted multi-backdoor implantation attack on llm-based agent. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 2890–2912. Association for Computational Linguistics, 2025.

[112] W. Zhu, Z. Xiang, W. Niu, and L. Guan. Metabreak: Jailbreaking online LLM services via special token manipulation. *CoRR*, abs/2510.10271, 2025.

[113] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models. *CoRR*, abs/2307.15043, 2023.

[114] W. Zou, R. Geng, B. Wang, and J. Jia. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. *CoRR*, abs/2402.07867, 2024.